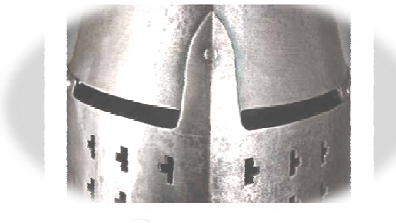


Cryptography and Network Security

Sixth Edition

by William Stallings



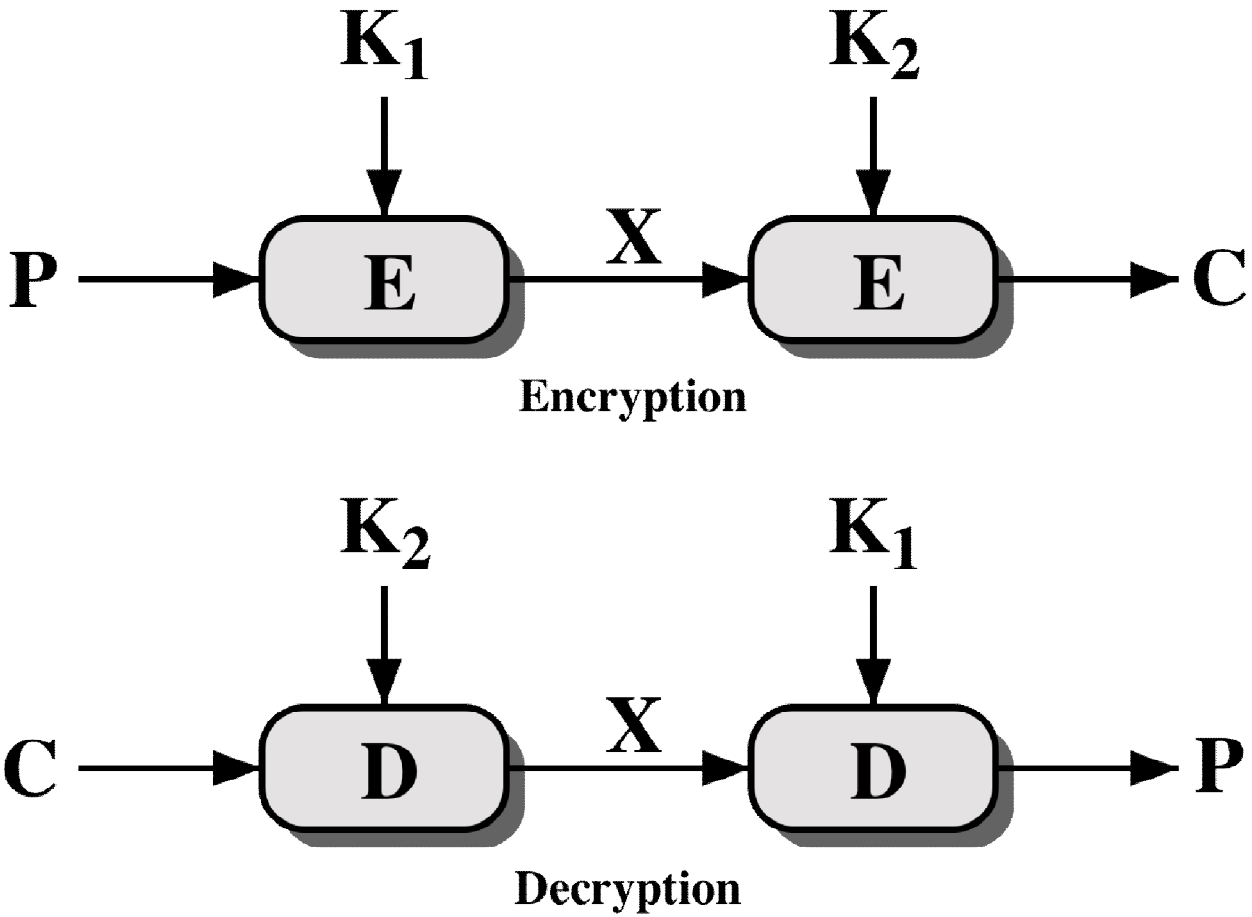
Chapter 6

Block Cipher Operation

“ Many savages at the present day regard their names as vital parts of themselves, and therefore take great pains to conceal their real names, lest these should give to evil-disposed persons a handle by which to injure their owners.”

— The Golden Bough,
Sir James George Frazer

Double DES



(a) Double Encryption

Meet-in-the-Middle Attack

The use of double DES results in a mapping that is not equivalent to a single DES encryption

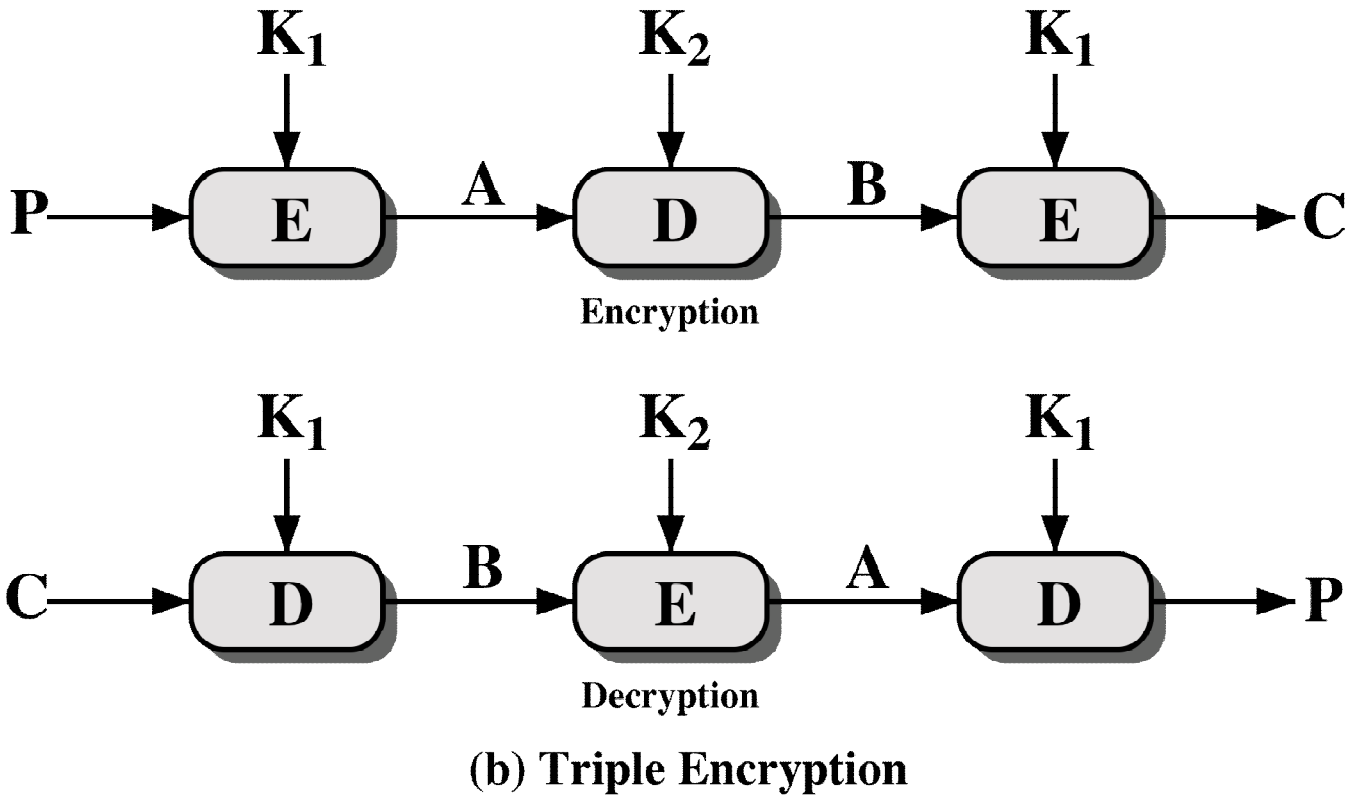


The meet-in-the-middle attack algorithm will attack this scheme and does not depend on any particular property of DES but will work against any block encryption cipher

Triple-DES with Two-Keys

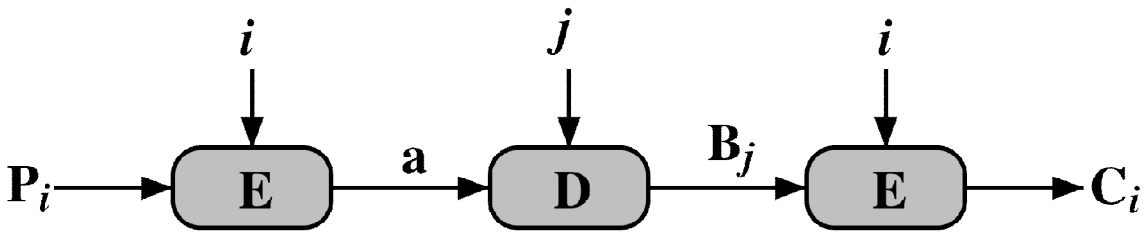
- Obvious counter to the meet-in-the-middle attack is to use three stages of encryption with three different keys
  - This raises the cost of the meet-in-the-middle attack to 2^{112}, which is beyond what is practical
  - Has the drawback of requiring a key length of 56 x 3 = 168 bits, which may be somewhat unwieldy
  - As an alternative Tuchman proposed a triple encryption method that uses only two keys
- 3DES with two keys is a relatively popular alternative to DES and has been adopted for use in the key management standards ANSI X9.17 and ISO 8732

Multiple Encryption

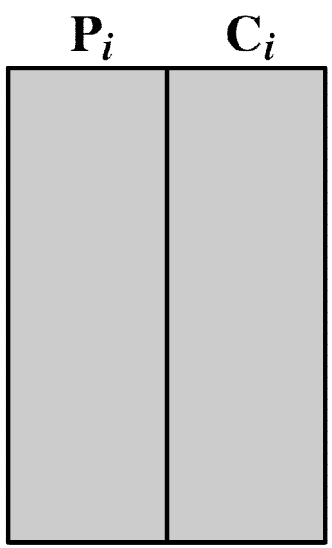


(b) Triple Encryption

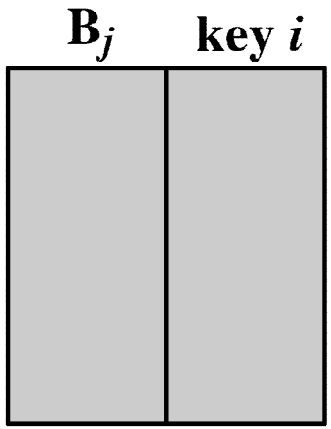
Figure 6.1 Multiple Encryption



(a) Two-key Triple Encryption with Candidate Pair of Keys



(b) Table of n known plaintext-ciphertext pairs, sorted on P



(c) Table of intermediate values and candidate keys

Figure 6.2 Known-Plaintext Attack on Triple DES

Triple DES with Three Keys

- Many researchers now feel that three-key 3DES is the preferred alternative

Three-key 3DES has an effective key length of 168 bits and is defined as:	• $C = E(K_3, D(K_2, E(K_1, P)))$
Backward compatibility with DES is provided by putting:	• $K_3 = K_2$ or $K_1 = K_2$

- A number of Internet-based applications have adopted three-key 3DES including PGP and S/MIME

Modes of Operation

- A technique for enhancing the effect of a cryptographic algorithm or adapting the algorithm for an application
- To apply a block cipher in a variety of applications, five *modes of operation* have been defined by NIST
  - The five modes are intended to cover a wide variety of applications of encryption for which a block cipher could be used
  - These modes are intended for use with any symmetric block cipher, including triple DES and AES

Table 6.1 Block Cipher Modes of Operation

Mode	Description	Typical Application
Electronic Codebook (ECB)	Each block of plaintext bits is encoded independently using the same key.	•Secure transmission of single values (e.g., an encryption key)
Cipher Block Chaining (CBC)	The input to the encryption algorithm is the XOR of the next block of plaintext and the preceding block of ciphertext.	•General-purpose block-oriented transmission •Authentication
Cipher Feedback (CFB)	Input is processed *s* bits at a time. Preceding ciphertext is used as input to the encryption algorithm to produce pseudorandom output, which is XORed with plaintext to produce next unit of ciphertext.	•General-purpose stream-oriented transmission •Authentication
Output Feedback (OFB)	Similar to CFB, except that the input to the encryption algorithm is the preceding encryption output, and full blocks are used.	•Stream-oriented transmission over noisy channel (e.g., satellite communication)
Counter (CTR)	Each block of plaintext is XORed with an encrypted counter. The counter is incremented for each subsequent block.	•General-purpose block-oriented transmission •Useful for high-speed requirements

Electronic Codebook Mode (ECB)

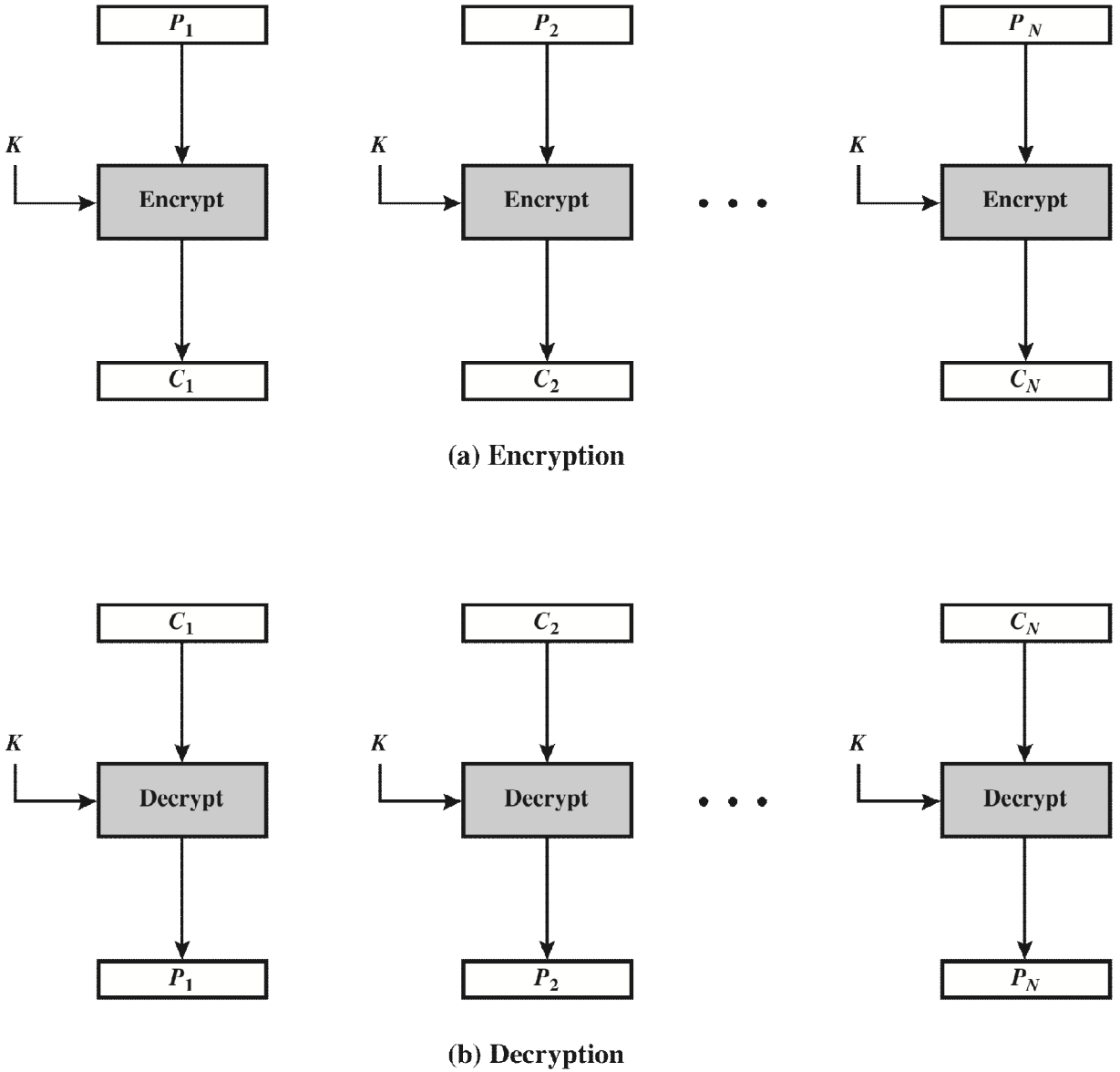


Figure 6.3 Electronic Codebook (ECB) Mode

Criteria and properties for evaluating and constructing block cipher modes of operation that are superior to ECB:



- Overhead
- Error recovery
- Error propagation
- Diffusion
- Security

Cipher Block Chaining (CBC)

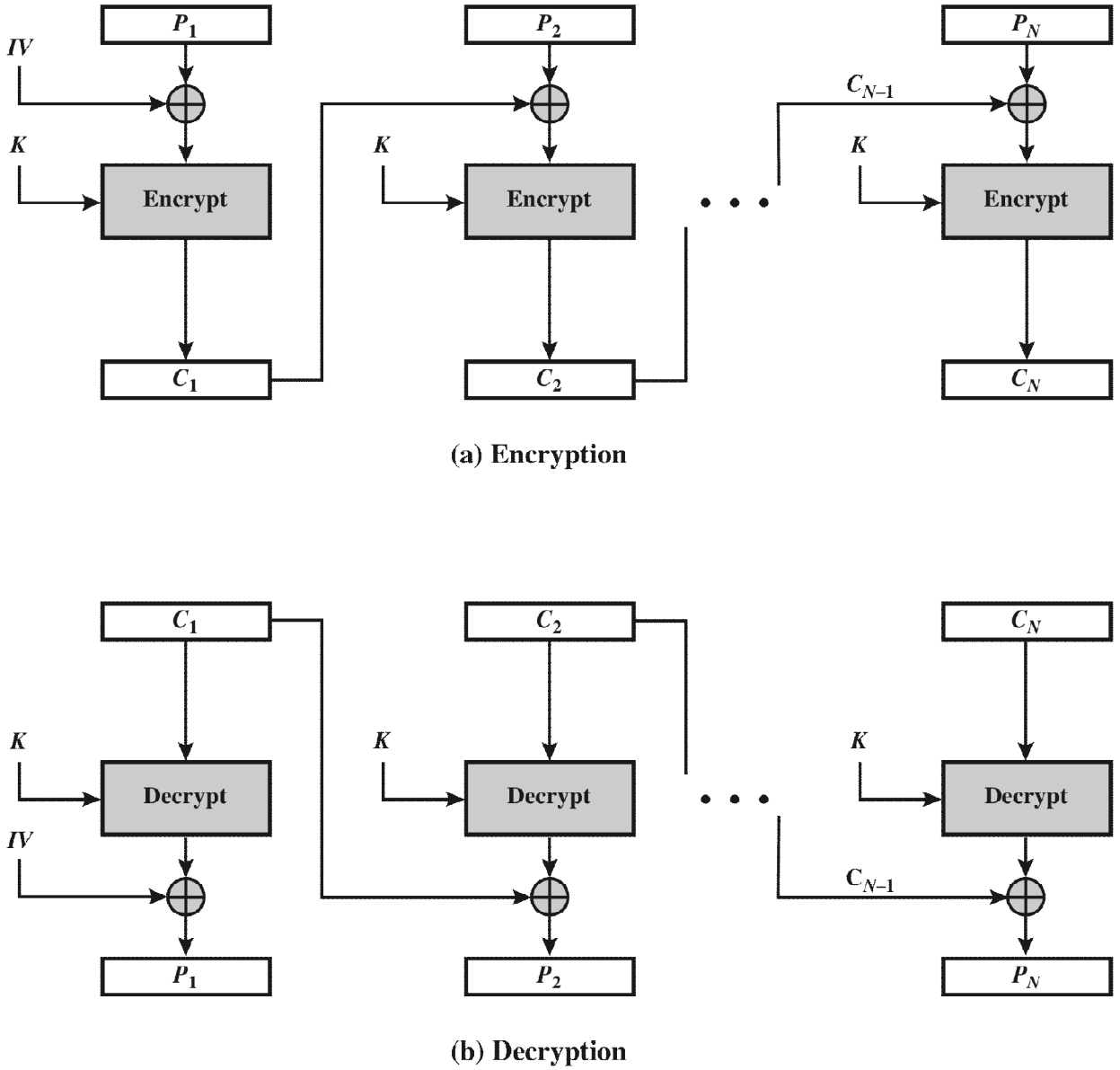
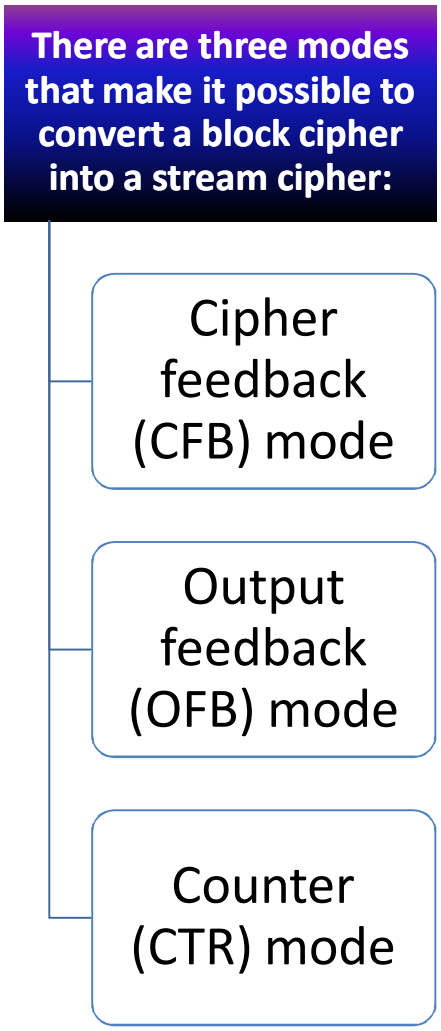


Figure 6.4 Cipher Block Chaining (CBC) Mode

Cipher Feedback Mode

- For AES, DES, or any block cipher, encryption is performed on a block of *b* bits
  - In the case of DES *b* = 64
  - In the case of AES *b* = 128



s-bit Cipher Feedback (CFB) Mode

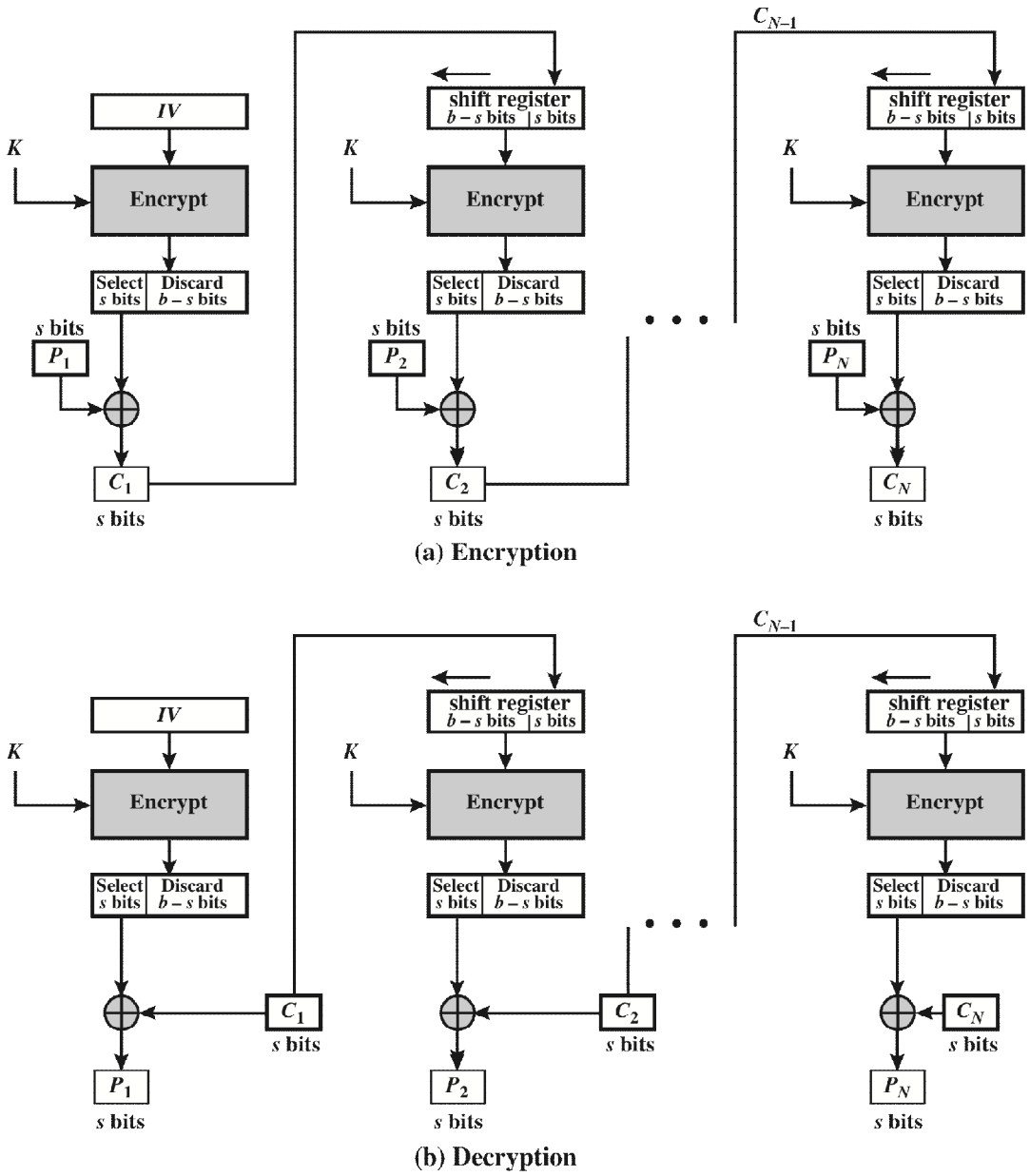
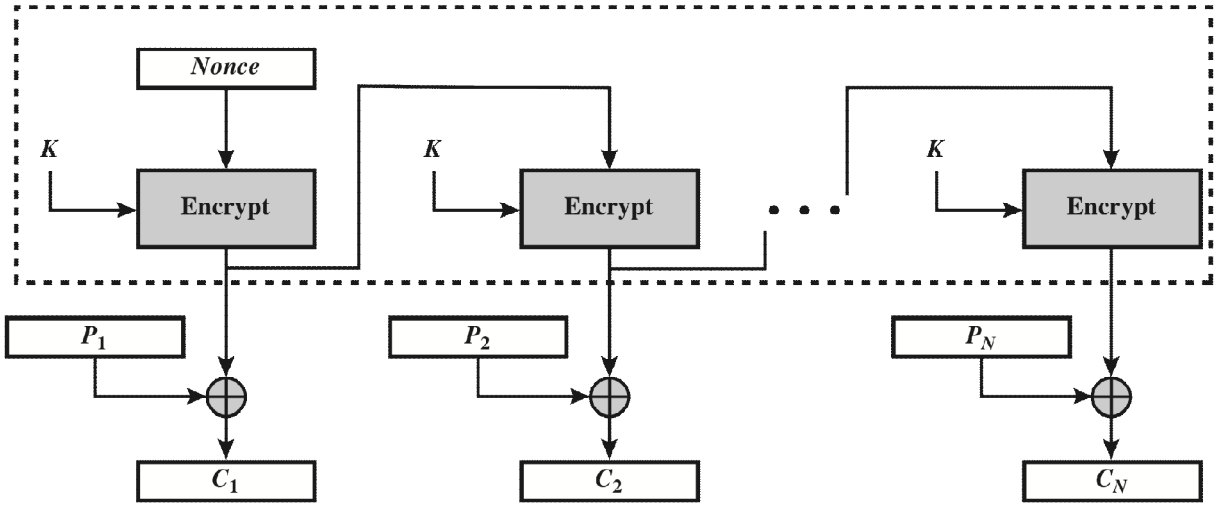
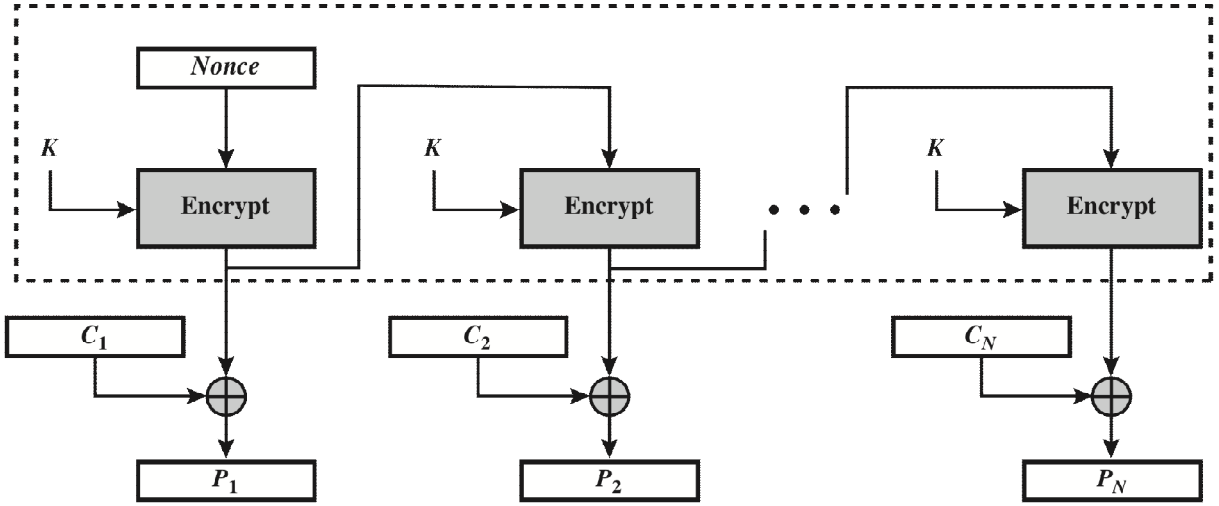


Figure 6.5 *s*-bit Cipher Feedback (CFB) Mode

Output Feedback (OFB) Mode



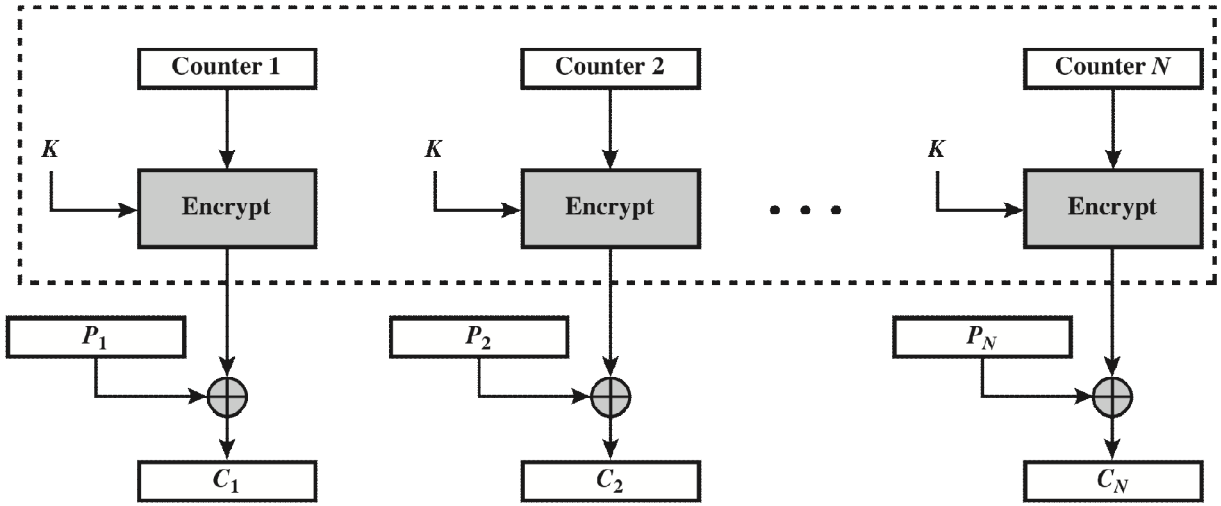
(a) Encryption



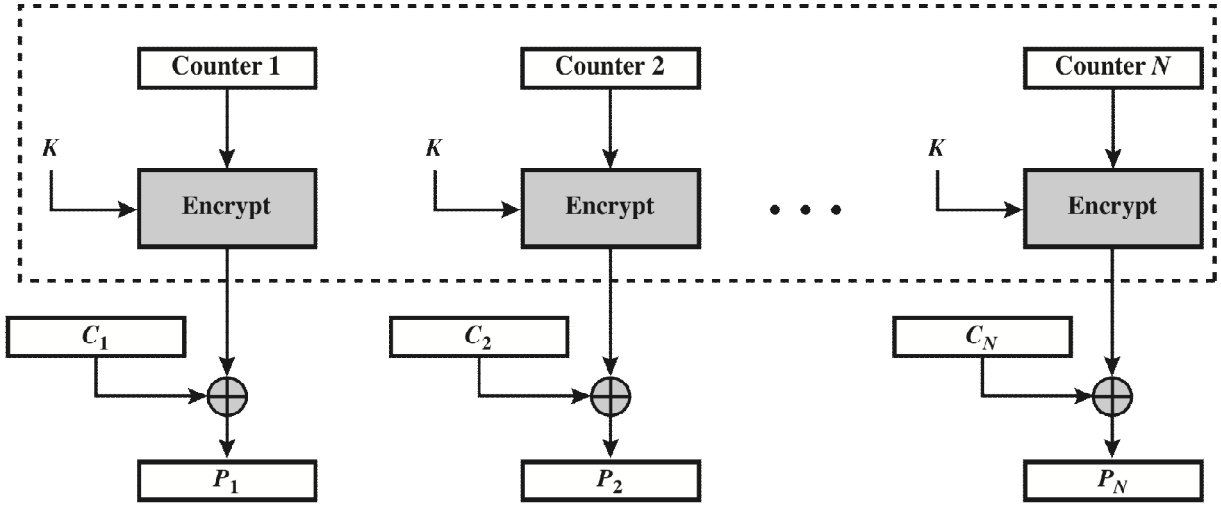
(b) Decryption

Figure 6.6 Output Feedback (OFB) Mode

Counter (CTR) Mode



(a) Encryption

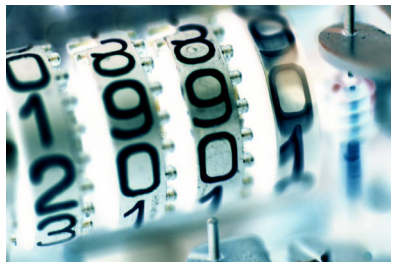


(b) Decryption

Figure 6.7 Counter (CTR) Mode

Advantages of CTR

- Hardware efficiency
- Software efficiency
- Preprocessing
- Random access
- Provable security
- Simplicity



Feedback Characteristics of Modes of Operation

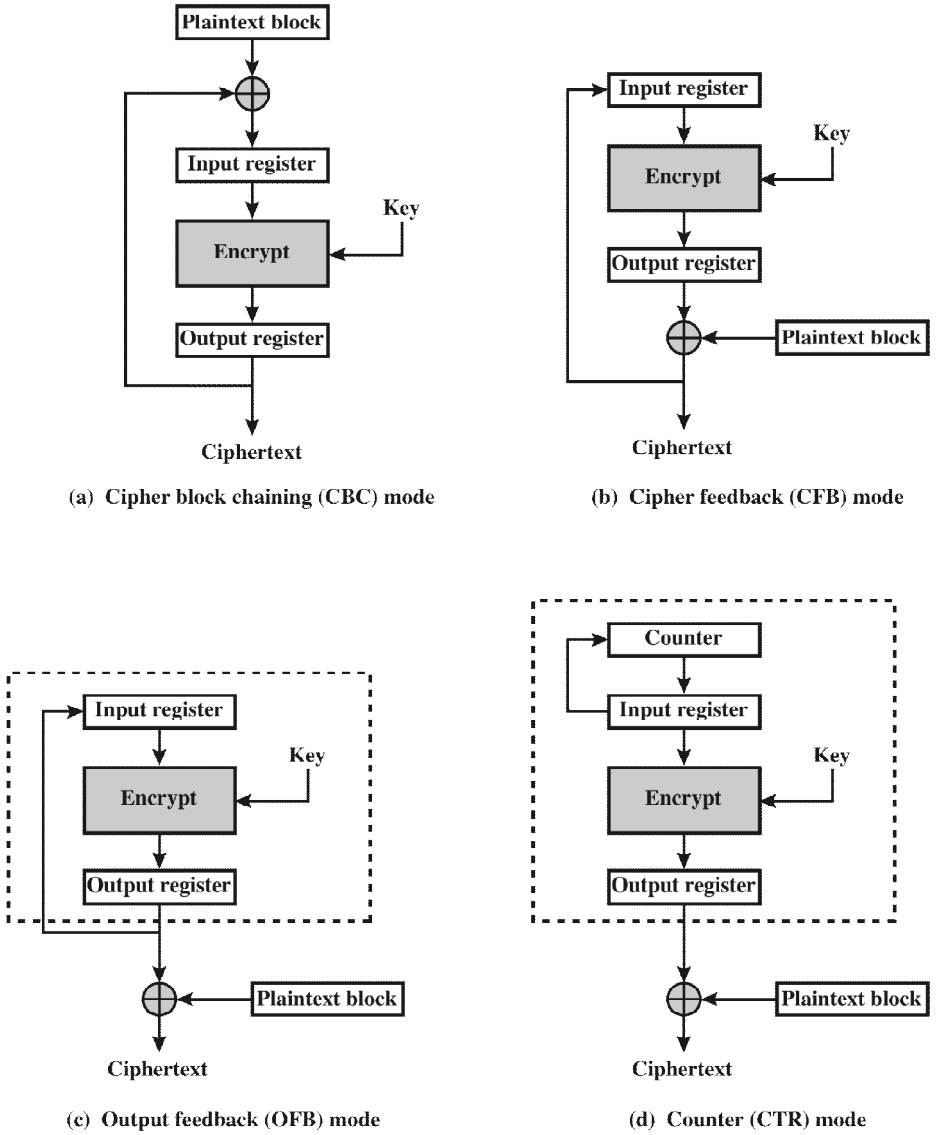


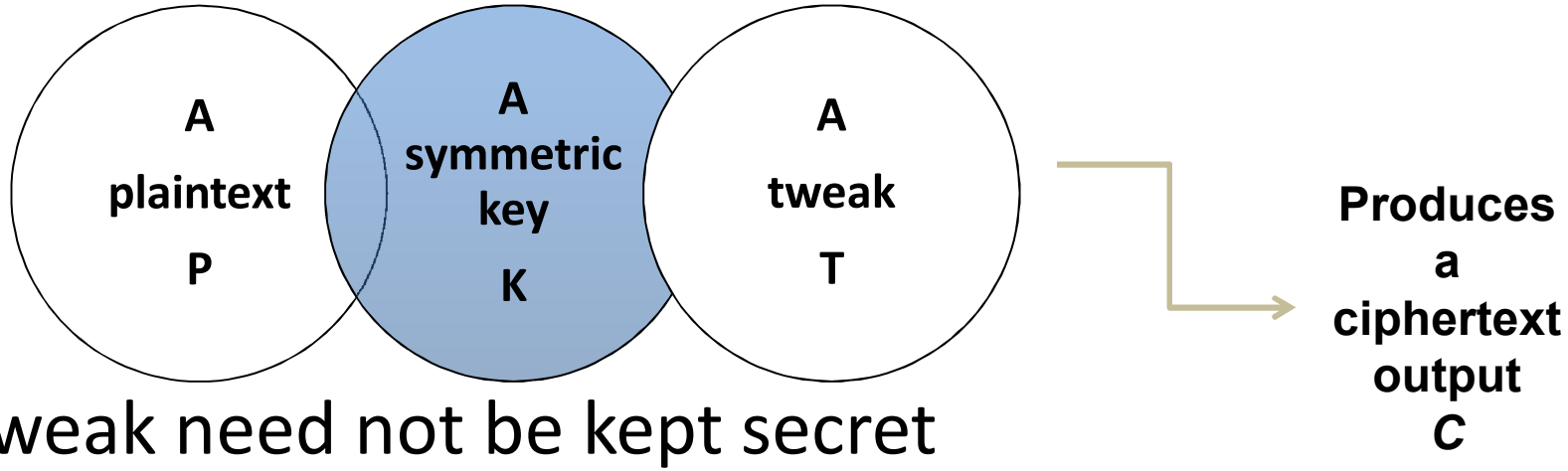
Figure 6.8 Feedback Characteristic of Modes of Operation

XTS-AES Mode for Block-Oriented Storage Devices

- Approved as an additional block cipher mode of operation by NIST in 2010
- Mode is also an IEEE Standard, IEEE Std 1619-2007
  - Standard describes a method of encryption for data stored in sector-based devices where the threat model includes possible access to stored data by the adversary
  - Has received widespread industry support

Tweakable Block Ciphers

- XTS-AES mode is based on the concept of a *tweakable block cipher*
- General structure:
  - Has three inputs:



- Tweak need not be kept secret
  - Purpose is to provide variability

Tweakable Block Cipher

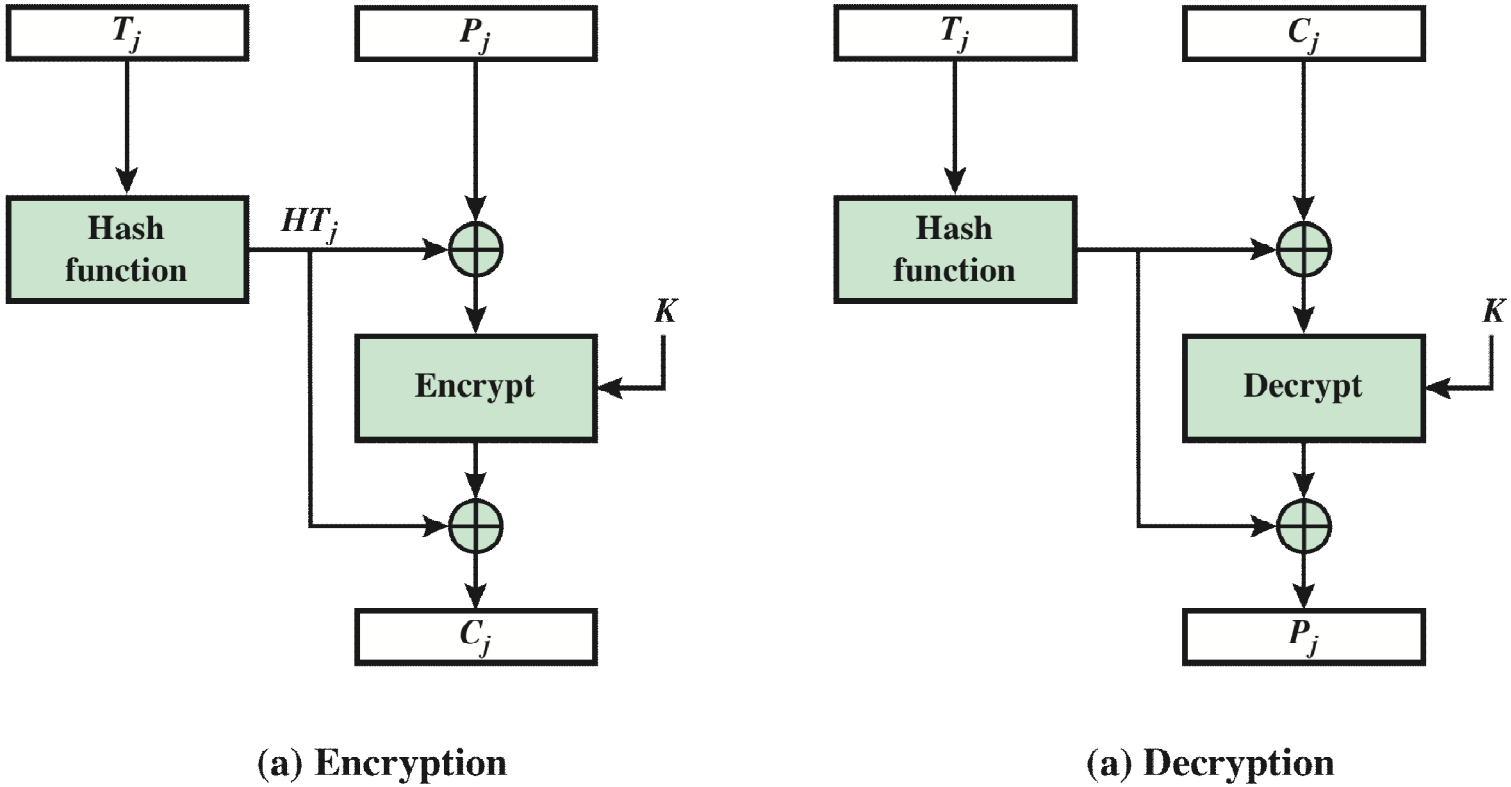
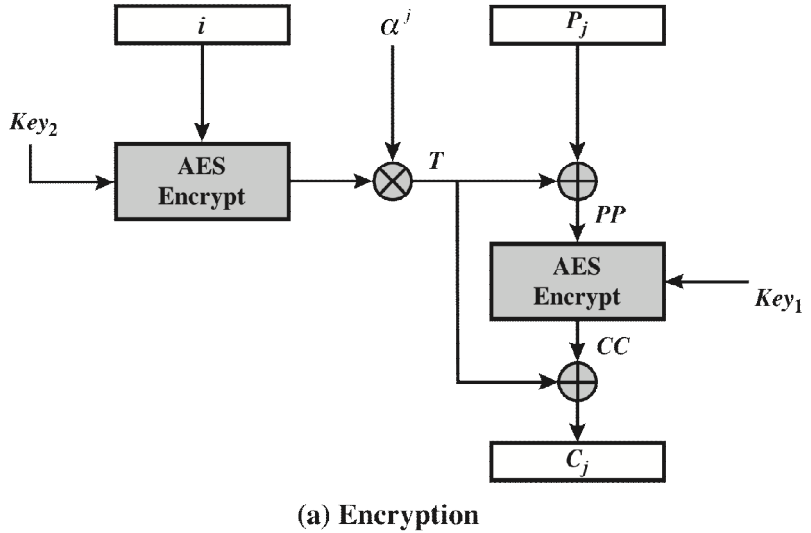
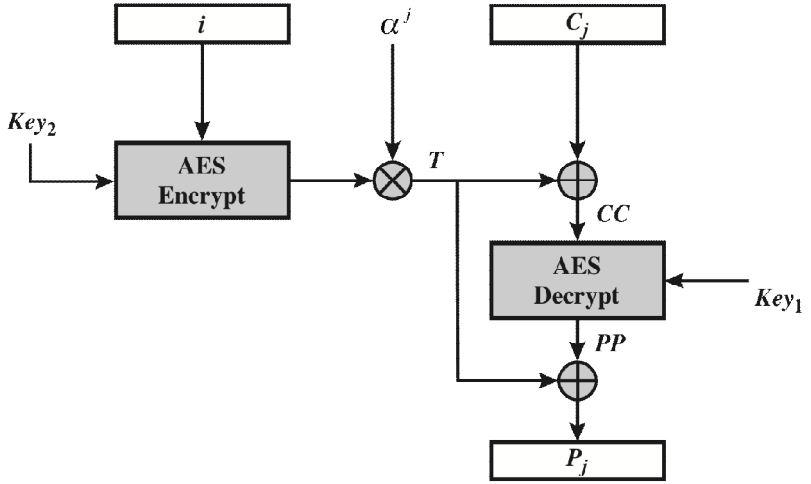


Figure 6.9 Tweakable Block Cipher

XTS-AES Operation on Single Block



(a) Encryption



(b) Decryption

Figure 6.10 XTS-AES Operation on Single Block

XTS–AES Mode

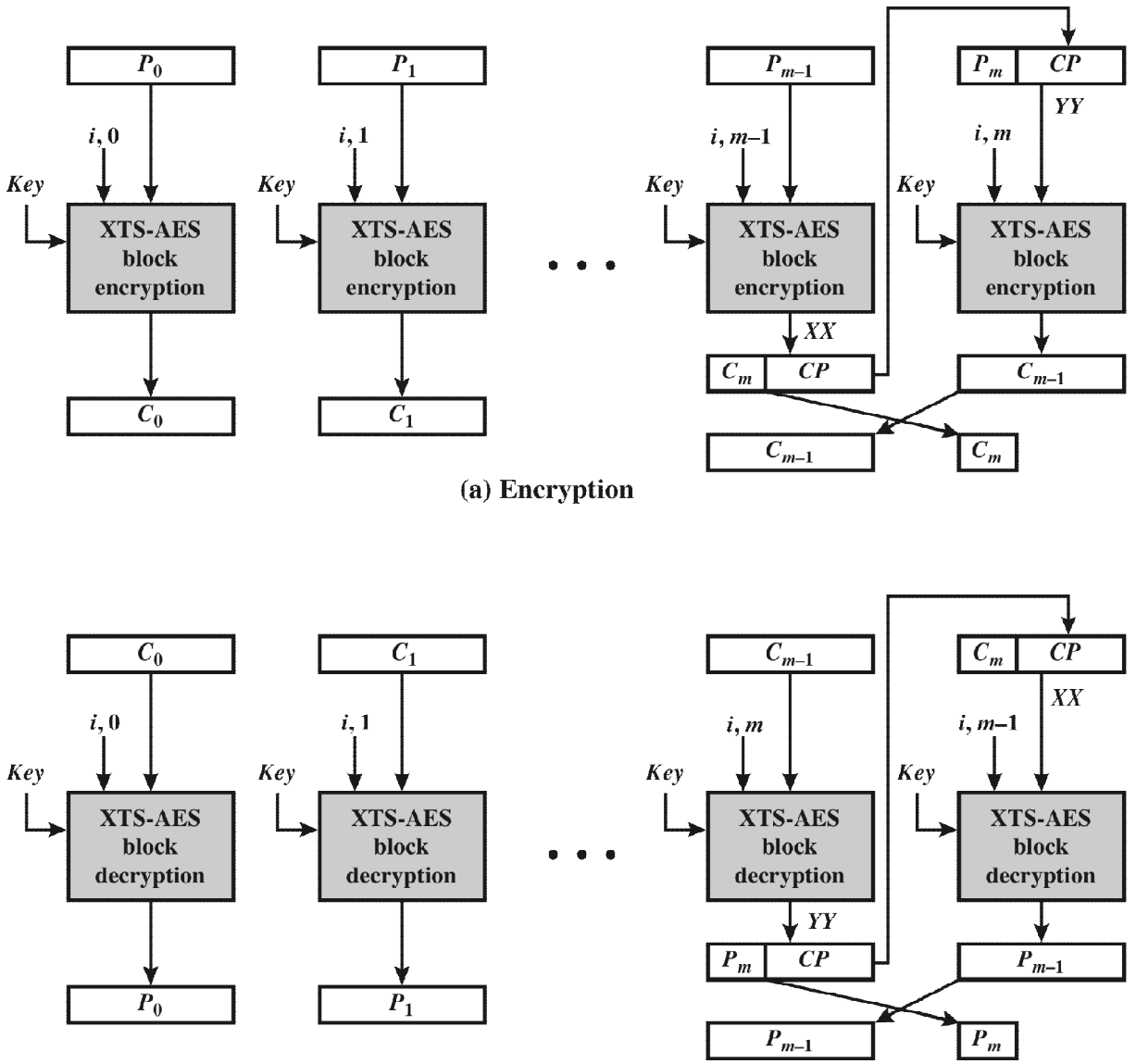
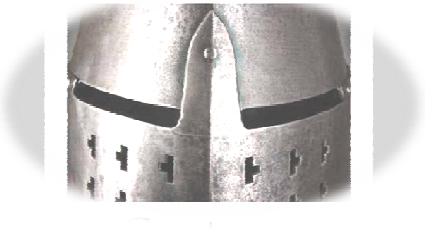


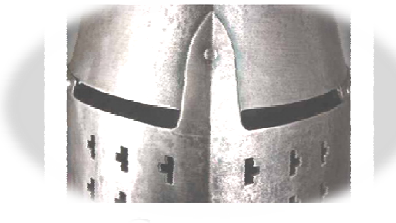
Figure 6.11 XTS-AES Mode

Summary

- Multiple encryption and triple DES
  - Double DES
  - Triple DES with two keys
  - Triple DES with three keys
- Electronic code book
- Cipher block chaining mode



- Cipher feedback mode
- Output feedback mode
- Counter mode
- XTS-AES mode for block-oriented storage devices
  - Storage encryption requirements
  - Operation on a single block
  - Operation on a sector



Chapter 7

Pseudorandom Number Generation and Stream Ciphers

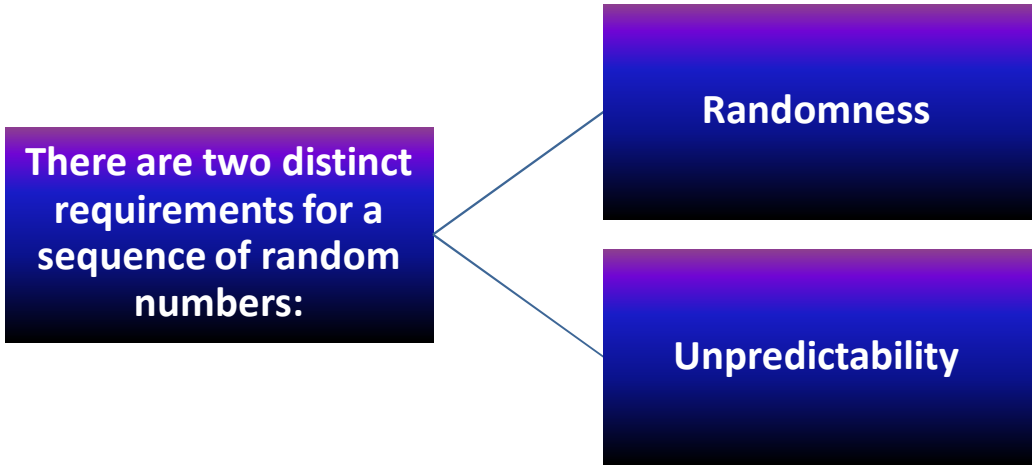
“The comparatively late rise of the theory of probability shows how hard it is to grasp, and the many paradoxes show clearly that we, as humans, lack a well grounded intuition in this matter.”

“In probability theory there is a great deal of art in setting up the model, in solving the problem, and in applying the results back to the real world actions that will follow.”

— The Art of Probability,
Richard Hamming

Random Numbers

- A number of network security algorithms and protocols based on cryptography make use of random binary numbers:
  - Key distribution and reciprocal authentication schemes
  - Session key generation
  - Generation of keys for the RSA public-key encryption algorithm
  - Generation of a bit stream for symmetric stream encryption



Randomness

- The generation of a sequence of allegedly random numbers being random in some well-defined statistical sense has been a concern

Two criteria are used to validate that a sequence of numbers is random:

Uniform distribution

- **The frequency of occurrence of ones and zeros should be approximately equal**

Independence

- **No one subsequence in the sequence can be inferred from the others**

Unpredictability

- The requirement is not just that the sequence of numbers be statistically random, but that the successive members of the sequence are unpredictable
- With “true” random sequences each number is statistically independent of other numbers in the sequence and therefore unpredictable
  - True random numbers have their limitations, such as inefficiency, so it is more common to implement algorithms that generate sequences of numbers that appear to be random
  - Care must be taken that an opponent not be able to predict future elements of the sequence on the basis of earlier elements

Pseudorandom Numbers

- Cryptographic applications typically make use of algorithmic techniques for random number generation
- These algorithms are deterministic and therefore produce sequences of numbers that are not statistically random
- If the algorithm is good, the resulting sequences will pass many tests of randomness and are referred to as *pseudorandom numbers*

True Random Number Generator (TRNG)

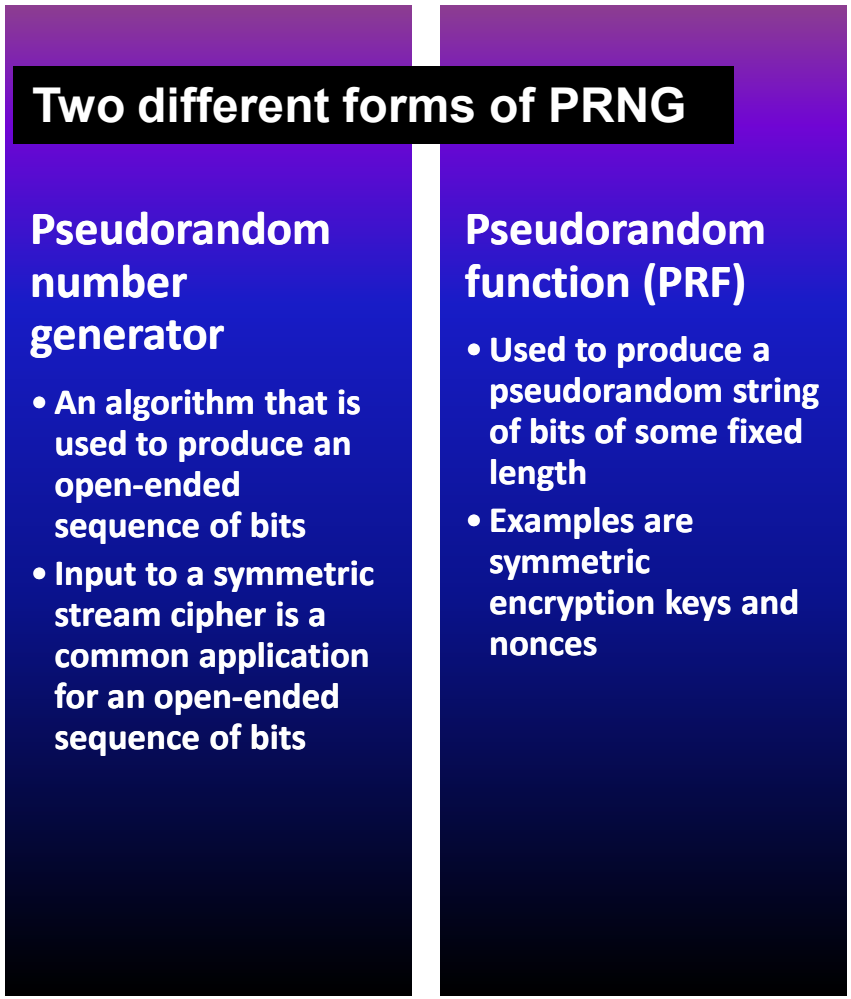
- Takes as input a source that is effectively random
- The source is referred to as an *entropy source* and is drawn from the physical environment of the computer
  - Includes things such as keystroke timing patterns, disk electrical activity, mouse movements, and instantaneous values of the system clock
  - The source, or combination of sources, serve as input to an algorithm that produces random binary output

- The TRNG may simply involve conversion of an analog source to a binary output

- The TRNG may involve additional processing to overcome any bias in the source

Pseudorandom Number Generator (PRNG)

- Takes as input a fixed value, called the *seed,* and produces a sequence of output bits using a deterministic algorithm
  - Quite often the seed is generated by a TRNG
- The output bit stream is determined solely by the input value or values, so an adversary who knows the algorithm and the seed can reproduce the entire bit stream
- Other than the number of bits produced there is no difference between a PRNG and a PRF



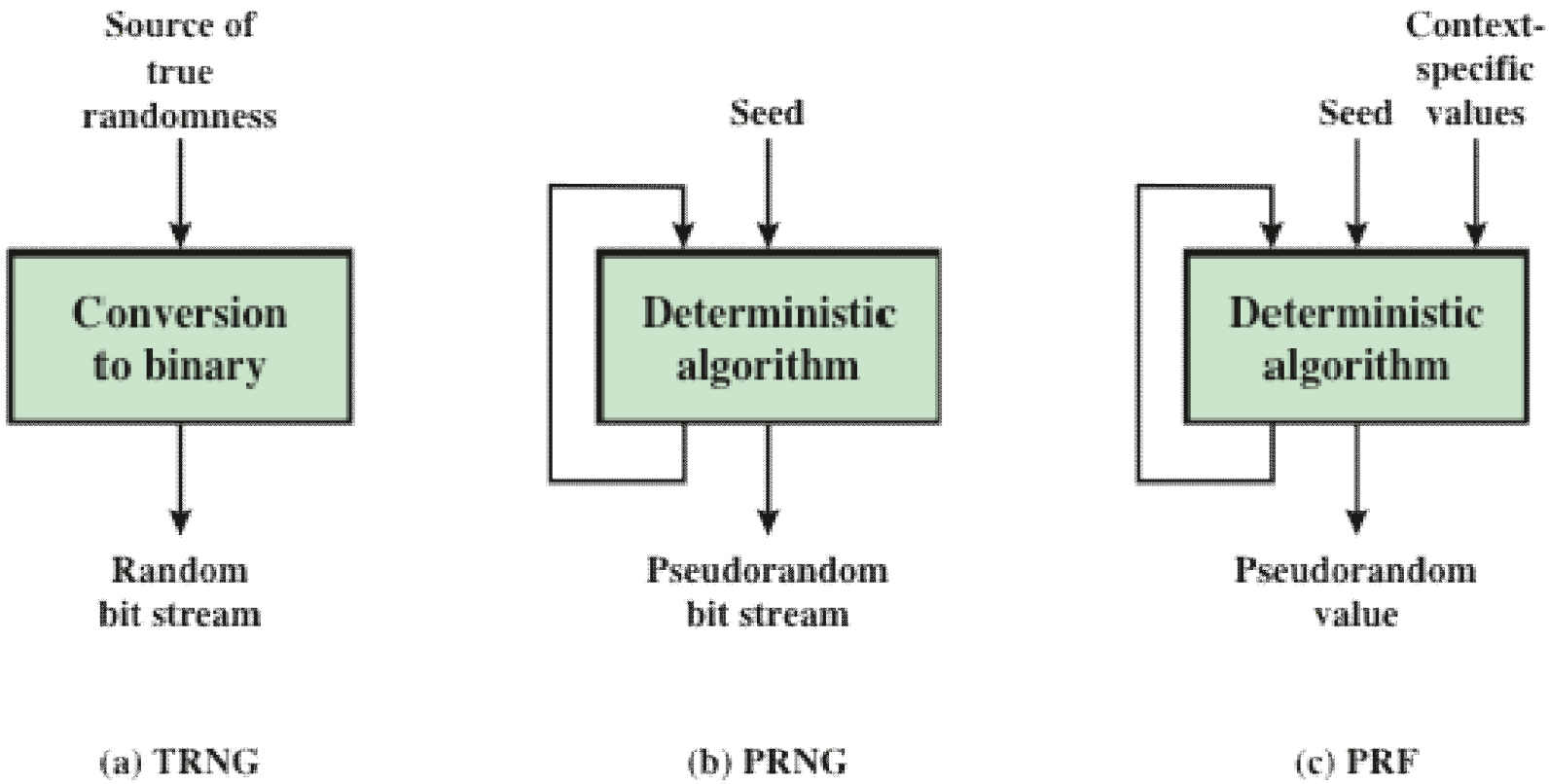


Figure 7.1 Random and Pseudorandom Number Generators

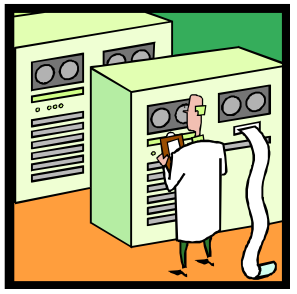
PRNG Requirements

- The basic requirement when a PRNG or PRF is used for a cryptographic application is that an adversary who does not know the seed is unable to determine the pseudorandom string
- The requirement for secrecy of the output of a PRNG or PRF leads to specific requirements in the areas of:
  - Randomness
  - Unpredictability
  - Characteristics of the seed



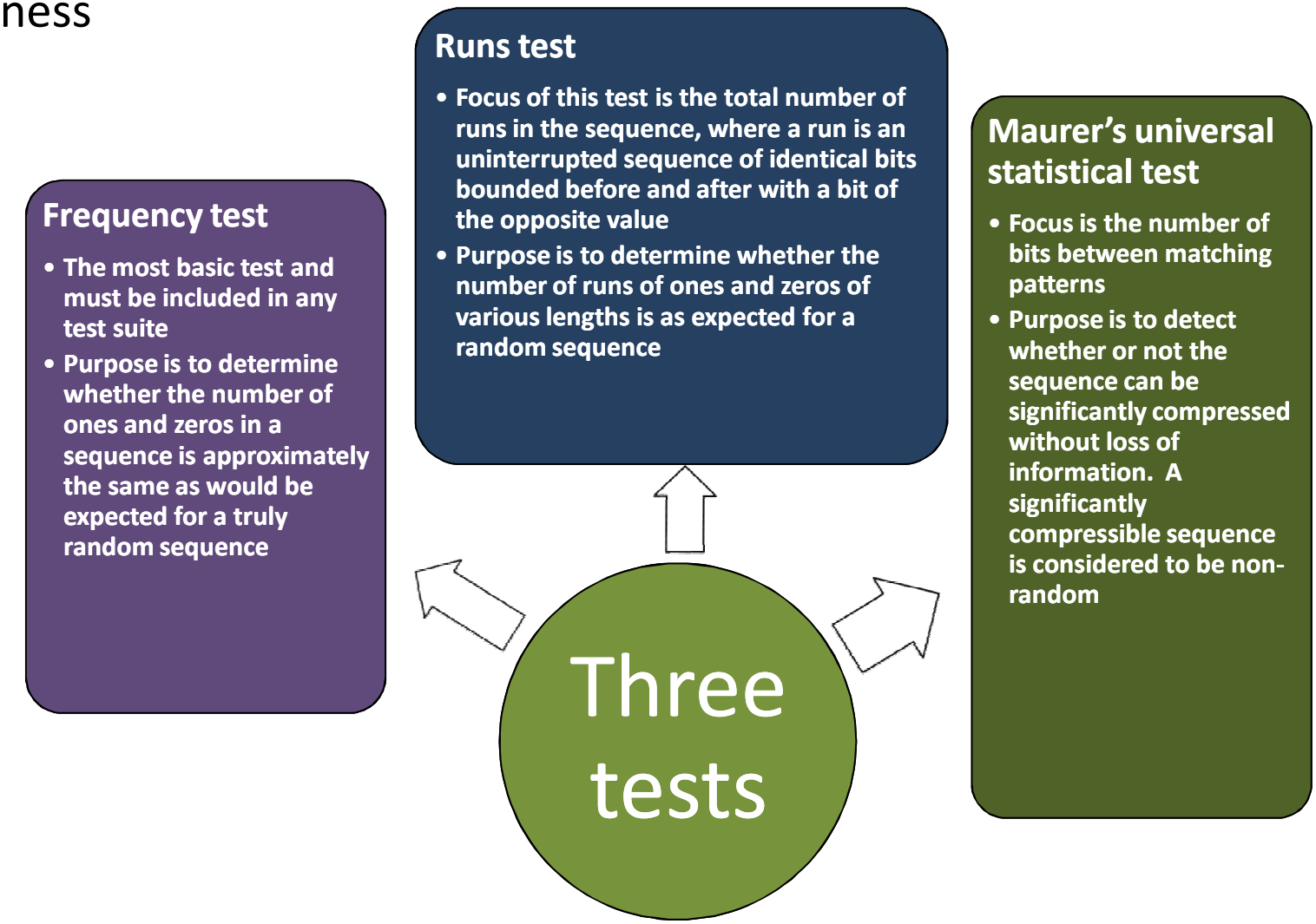
Randomness

- The generated bit stream needs to appear random even though it is deterministic
- There is no single test that can determine if a PRNG generates numbers that have the characteristic of randomness
  - If the PRNG exhibits randomness on the basis of multiple tests, then it can be assumed to satisfy the randomness requirement
- NIST SP 800-22 specifies that the tests should seek to establish three characteristics:
  - Uniformity
  - Scalability
  - Consistency



Randomness Tests

- SP 800-22 lists 15 separate tests of randomness



Unpredictability

- A stream of pseudorandom numbers should exhibit two forms of unpredictability:
- Forward unpredictability
  - If the seed is unknown, the next output bit in the sequence should be unpredictable in spite of any knowledge of previous bits in the sequence
- Backward unpredictability
  - It should not be feasible to determine the seed from knowledge of any generated values. No correlation between a seed and any value generated from that seed should be evident; each element of the sequence should appear to be the outcome of an independent random event whose probability is 1/2
- The same set of tests for randomness also provides a test of unpredictability
  - A random sequence will have no correlation with a fixed value (the seed)

Seed Requirements

- The seed that serves as input to the PRNG must be secure and unpredictable
- The seed itself must be a random or pseudorandom number
- Typically the seed is generated by TRNG



Generation of Seed Input to PRNG

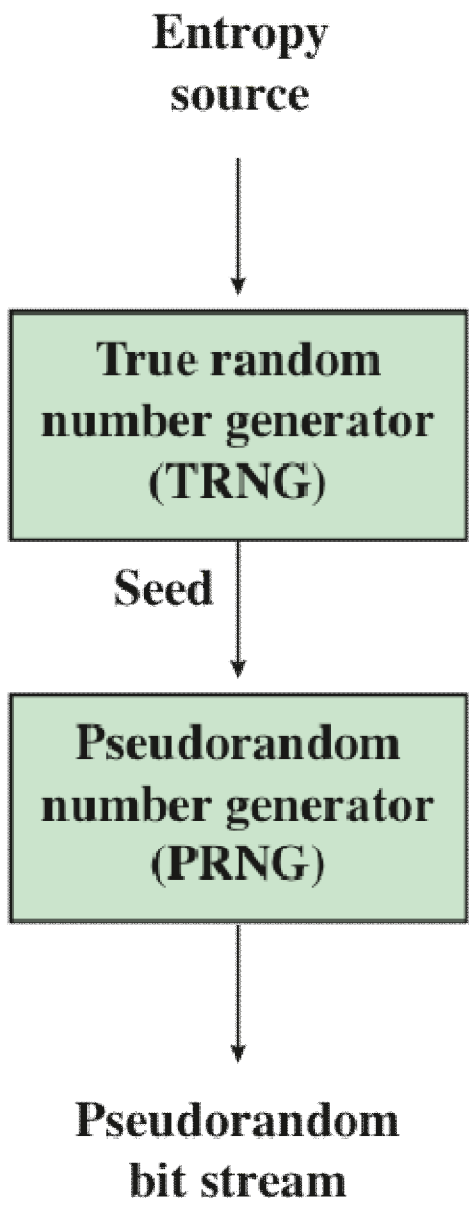


Figure 7.2 Generation of Seed Input to PRNG

Algorithm Design

- Algorithms fall into two categories:
  - Purpose-built algorithms
    - Algorithms designed specifically and solely for the purpose of generating pseudorandom bit streams
  - Algorithms based on existing cryptographic algorithms
    - Have the effect of randomizing input data

Three broad categories of cryptographic algorithms are commonly used to create PRNGs:

- Symmetric block ciphers
- Asymmetric ciphers
- Hash functions and message authentication codes

Linear Congruential Generator

- An algorithm first proposed by Lehmer that is parameterized with four numbers:

m	the modulus	$m > 0$
a	the multiplier	$0 < a < m$
c	the increment	$0 \leq c < m$
X_0	the starting value, or seed	$0 \leq X_0 < m$

- The sequence of random numbers $\{X_n\}$ is obtained via the following iterative equation:

$$X_{n+1} = (aX_n + c) \bmod m$$

- If m , a , c , and X_0 are integers, then this technique will produce a sequence of integers with each integer in the range $0 \leq X_n < m$
- The selection of values for a , c , and m is critical in developing a good random number generator

Blum Blum Shub (BBS) Generator

- Has perhaps the strongest public proof of its cryptographic strength of any purpose-built algorithm
- Referred to as a *cryptographically secure pseudorandom bit generator* (CSPRBG)
  - A CSPRBG is defined as one that passes the *next-bit-test* if there is not a polynomial-time algorithm that, on input of the first k bits of an output sequence, can predict the $(k + 1)$st bit with probability significantly greater than 1/2
- The security of BBS is based on the difficulty of factoring n

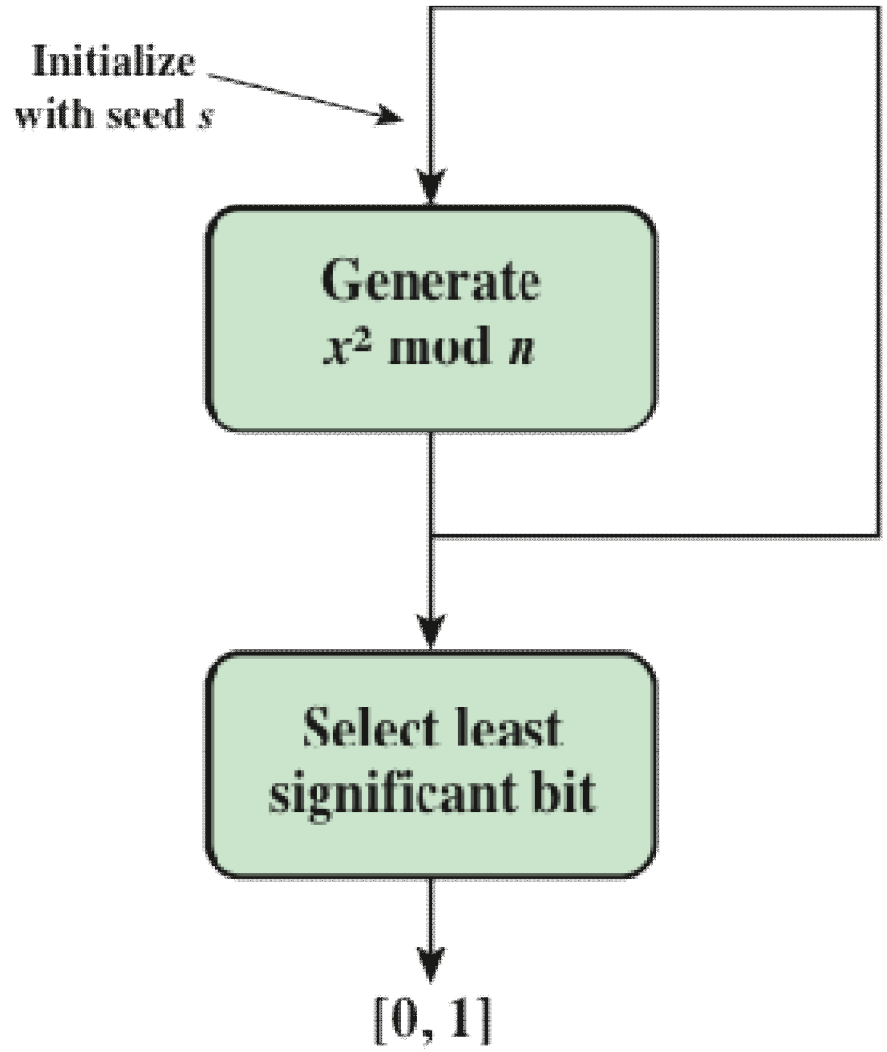


Figure 7.3 Blum Blum Shub Block Diagram

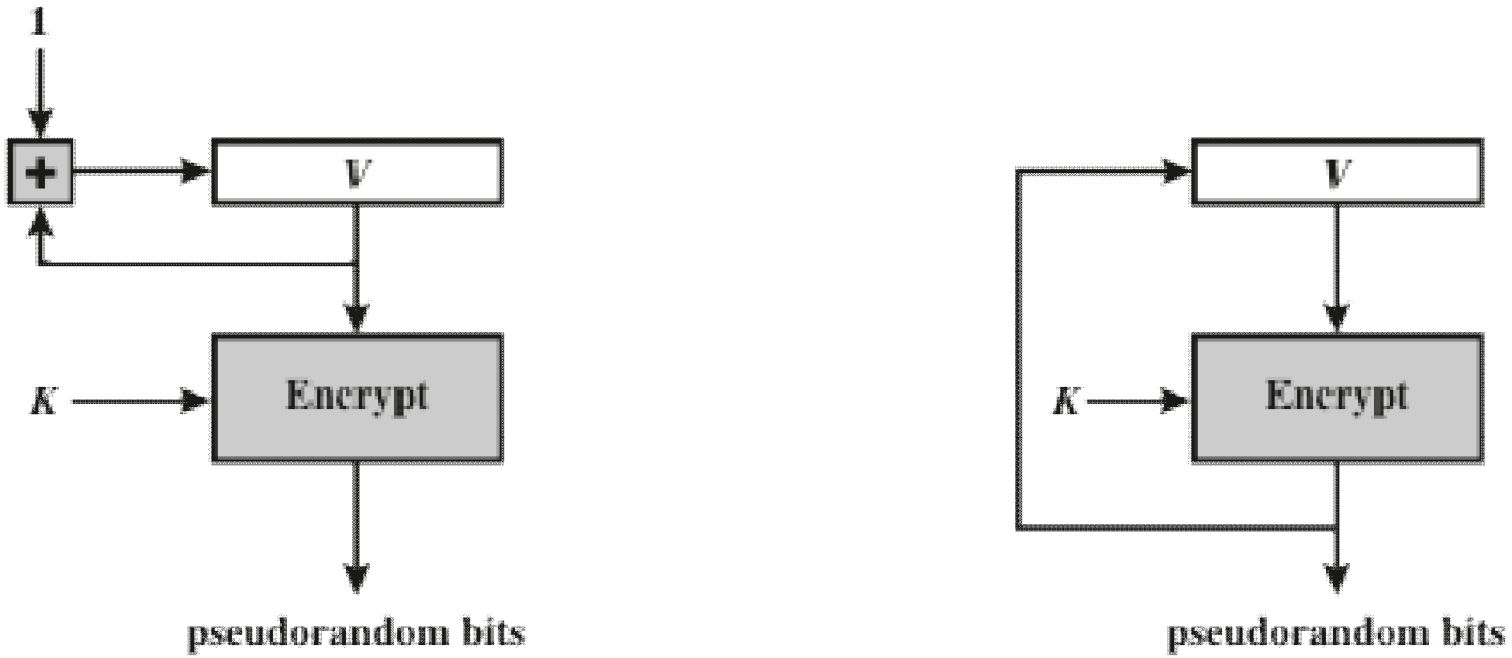
Table 7.1

Example Operation of BBS Generator

i	X_i	B_i
0	20749	
1	143135	1
2	177671	1
3	97048	0
4	89992	0
5	174051	1
6	80649	1
7	45663	1
8	69442	0
9	186894	0
10	177046	0
11	137922	0
12	123175	1
13	8630	0
14	114386	0
15	14863	1
16	133015	1
17	106065	1
18	45870	0
19	137171	1
20	48060	0

PRNG Using Block Cipher Modes of Operation

- Two approaches that use a block cipher to build a PNRG have gained widespread acceptance:
  - CTR mode
    - Recommended in NIST SP 800-90, ANSI standard X.82, and RFC 4086
  - OFB mode
    - Recommended in X9.82 and RFC 4086



(a) CTR Mode

(b) OFB Mode

Figure 7.4 PRNG Mechanisms Based on Block Ciphers

Table 7.2

Output Block	Fraction of One Bits	Fraction of Bits that Match with Preceding Block
1786f4c7ff6e291dbdfdd90ec3453176	0.57	—
5e17b22b14677a4d66890f87565eae64	0.51	0.52
fd18284ac82251dfb3aa62c326cd46cc	0.47	0.54
c8e545198a758ef5dd86b41946389bd5	0.50	0.44
fe7bae0e2301954296 2e2c52d215a2e3	0.47	0.48
14fdf5ec99469598ae0379472803accd	0.49	0.52
6aeca972e5a3ef17bd1a1b775fc8b929	0.57	0.48
f7e97badf359d128f00d9b4ae323db64	0.55	0.45

Example Results for PRNG Using OFB

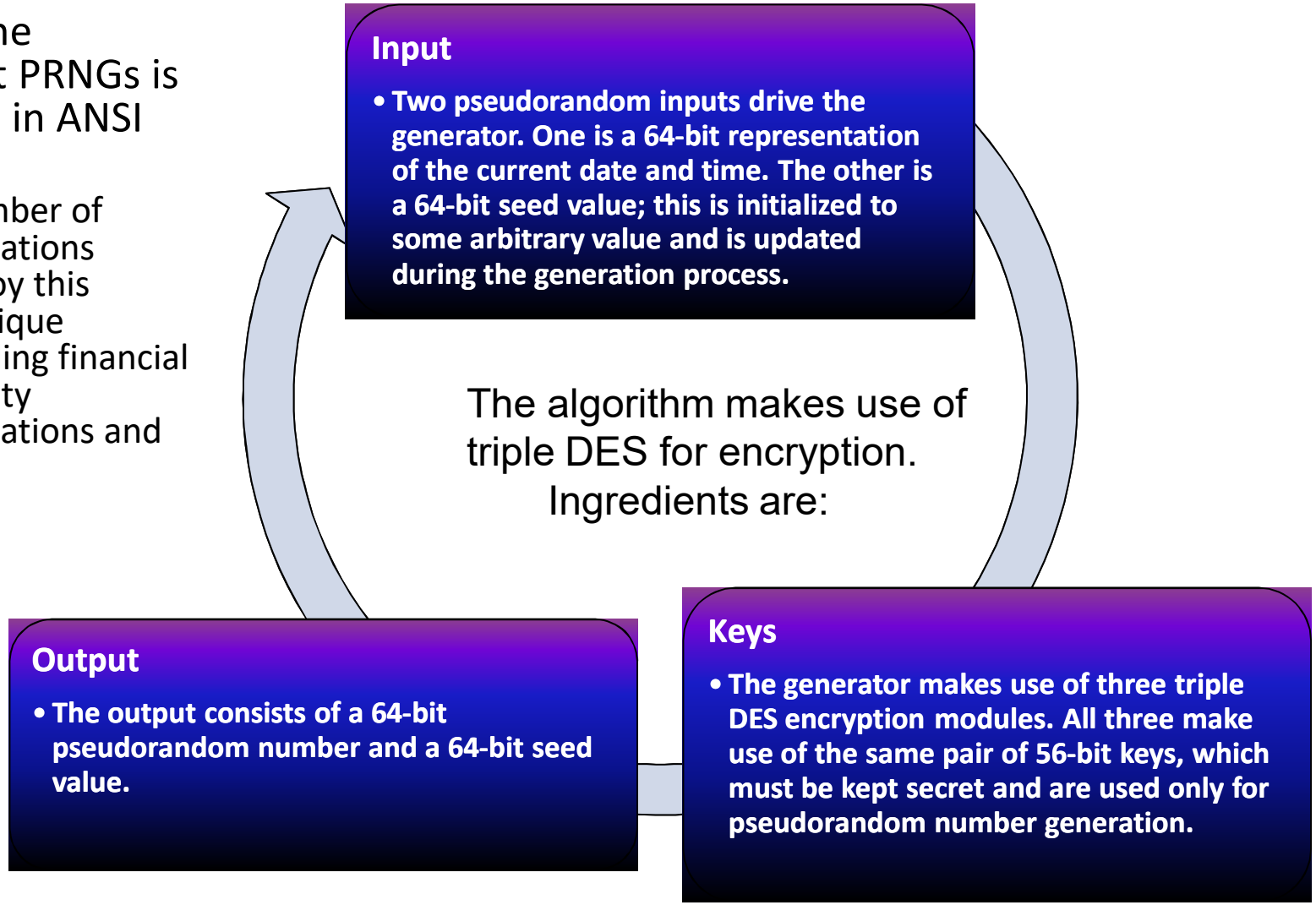
Table 7.3

Output Block	Fraction of One Bits	Fraction of Bits that Match with Preceding Block
1786f4c7ff6e291dbdfdd90ec3453176	0.57	—
60809669a3e092a01b463472fdcae420	0.41	0.41
d4e6e170b46b0573eecf88ee39bff33d	0.59	0.45
5f8fcfc5deca18ea24c785d7fadc76f8	0.59	0.52
90e63ed27bb07868c753545bdd57ee28	0.53	0.52
0125356fdf4a17f747c7833695c52235	0.50	0.47
f4be2d179b0f2548fd740c8fc7c81990	0.51	0.48
1151fc48f90eebac658a3911515c3c66	0.47	0.45

Example Results for PRNG Using CTR

ANSI X9.17 PRNG

- One of the strongest PRNGs is specified in ANSI X9.17
  - A number of applications employ this technique including financial security applications and PGP



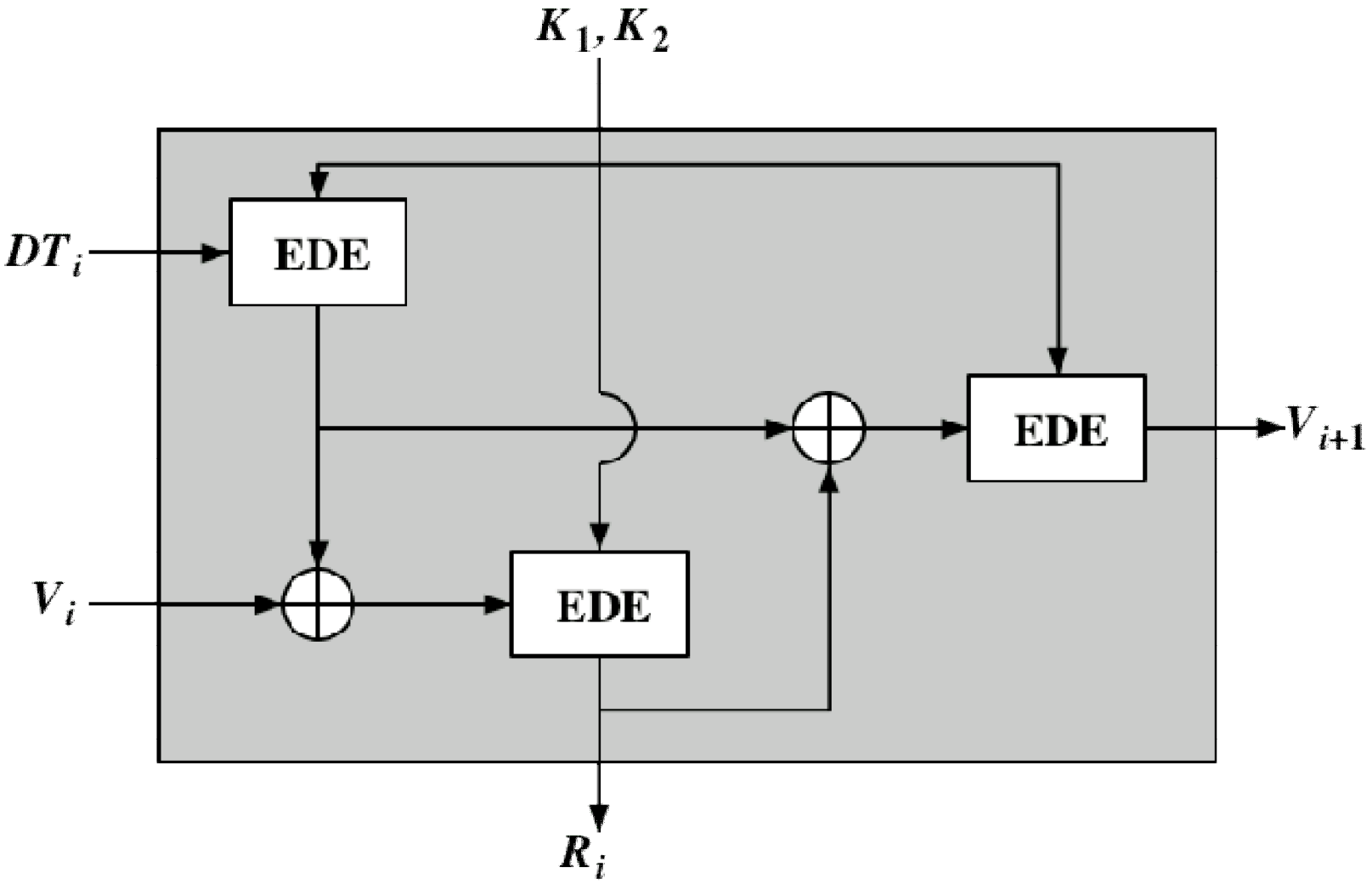


Figure 7.5 ANSI X9.17 Pseudorandom Number Generator

NIST CTR_DRBG

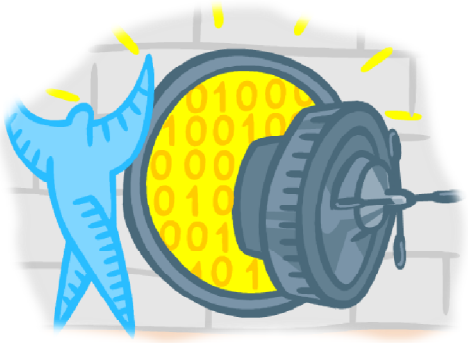
- Counter mode-deterministic random bit generator
- PRNG defined in NIST SP 800-90 based on the CTR mode of operation
- Is widely implemented and is part of the hardware random number generator implemented on all recent Intel processor chips
- DRBG assumes that an entropy source is available to provide random bits
  - Entropy is an information theoretic concept that measures unpredictability or randomness
- The encryption algorithm used in the DRBG may be 3DES with three keys or AES with a key size of 128, 192, or 256 bits

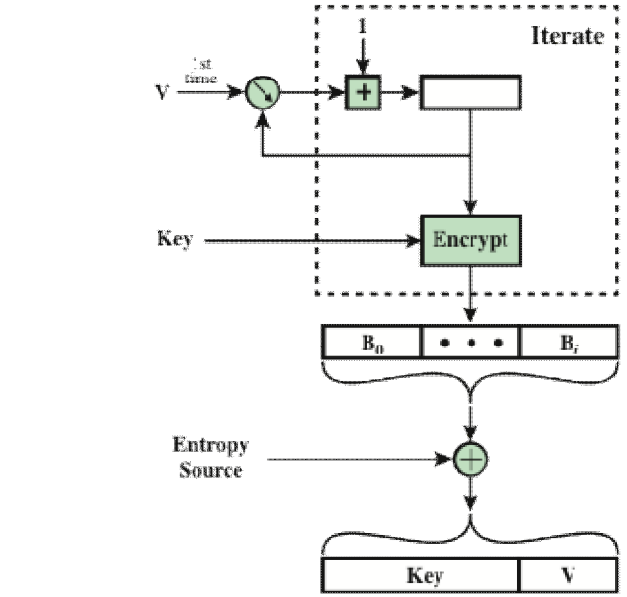
Table 7.4

	3DES	AES-128	AES-192	AES-256
outlen	64	128	128	128
keylen	168	128	192	256
seedlen	232	256	320	384
reseed_interval	$\leq 2^{32}$	$\leq 2^{48}$	$\leq 2^{48}$	$\leq 2^{48}$

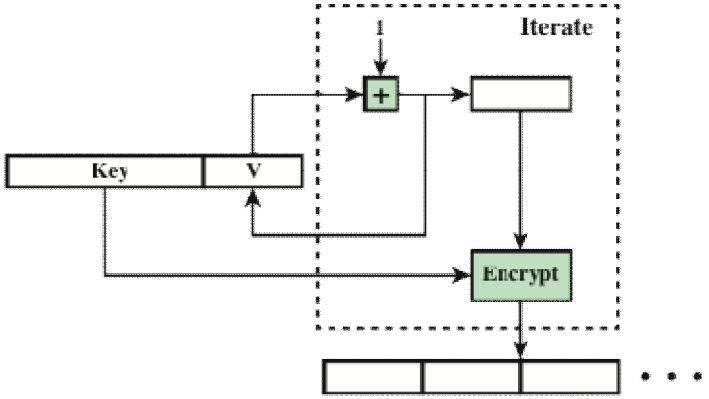
CTR_DRBG Parameters

CTR_DRBG Functions





(a) Initialize and update function



(b) Generate function

Figure 7.6 CTR_DRBG Functions

Stream Ciphers

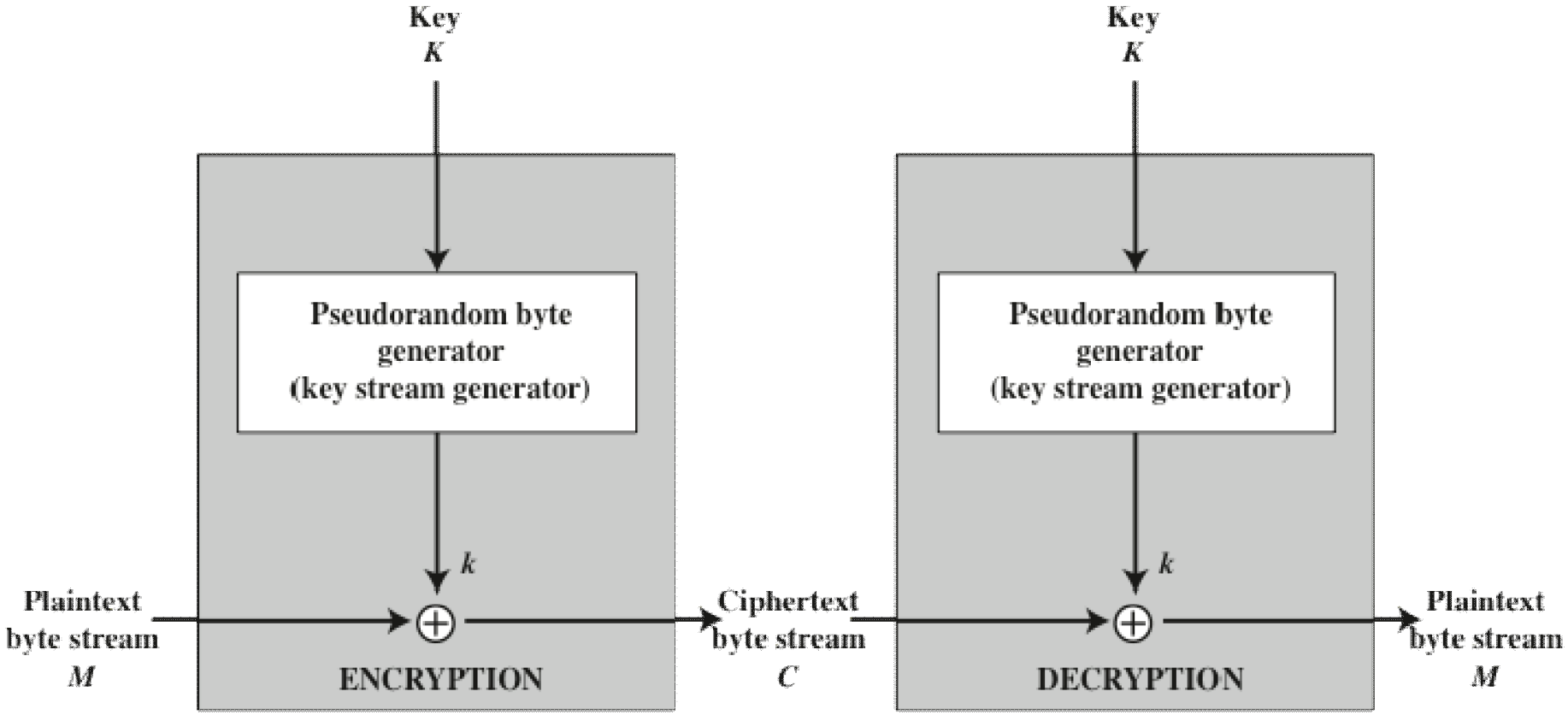


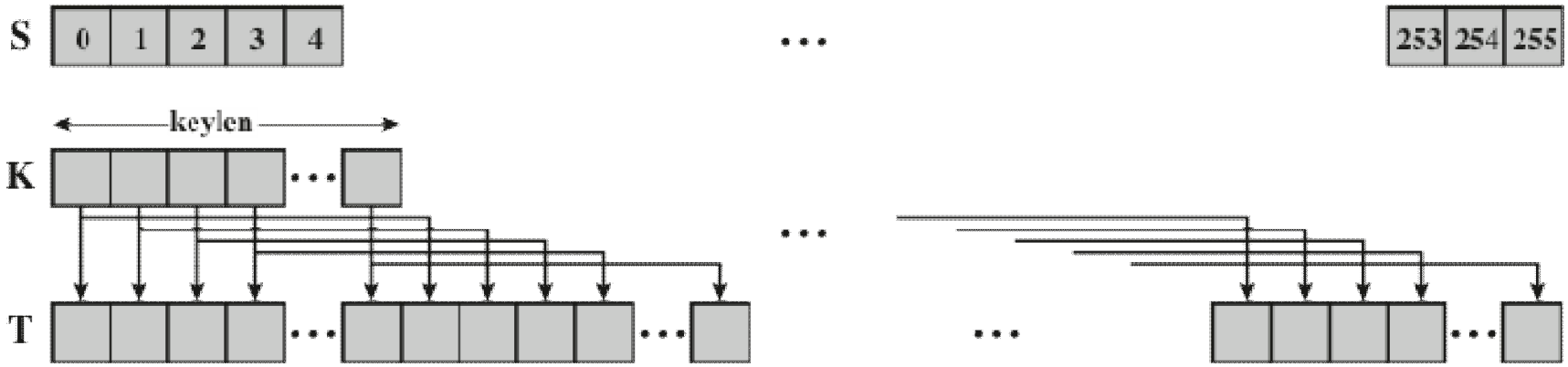
Figure 7.7 Stream Cipher Diagram

Stream Cipher Design Considerations

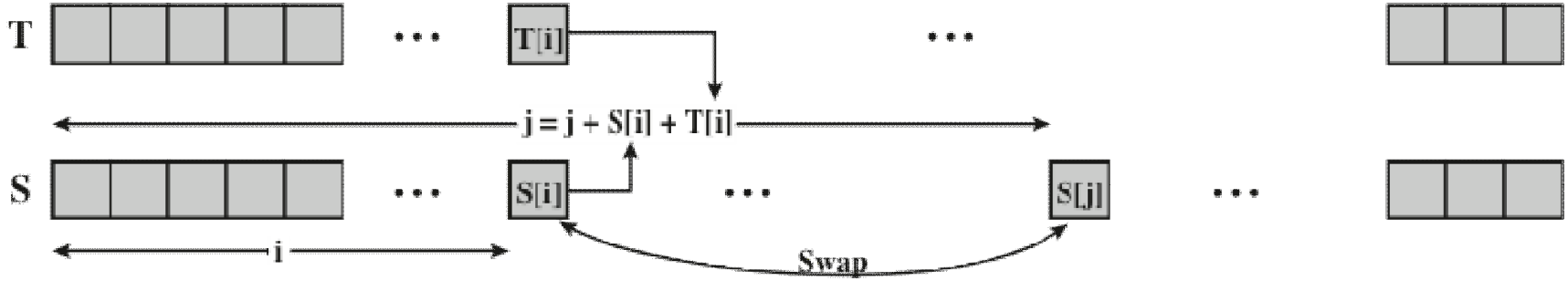
The encryption sequence should have a large period	• A pseudorandom number generator uses a function that produces a deterministic stream of bits that eventually repeats; the longer the period of repeat the more difficult it will be to do cryptanalysis
The keystream should approximate the properties of a true random number stream as close as possible	• There should be an approximately equal number of 1s and 0s • If the keystream is treated as a stream of bytes, then all of the 256 possible byte values should appear approximately equally often
A key length of at least 128 bits is desirable	• The output of the pseudorandom number generator is conditioned on the value of the input key • The same considerations that apply to block ciphers are valid
With a properly designed pseudorandom number generator a stream cipher can be as secure as a block cipher of comparable key length	• A potential advantage is that stream ciphers that do not use block ciphers as a building block are typically faster and use far less code than block ciphers

RC4

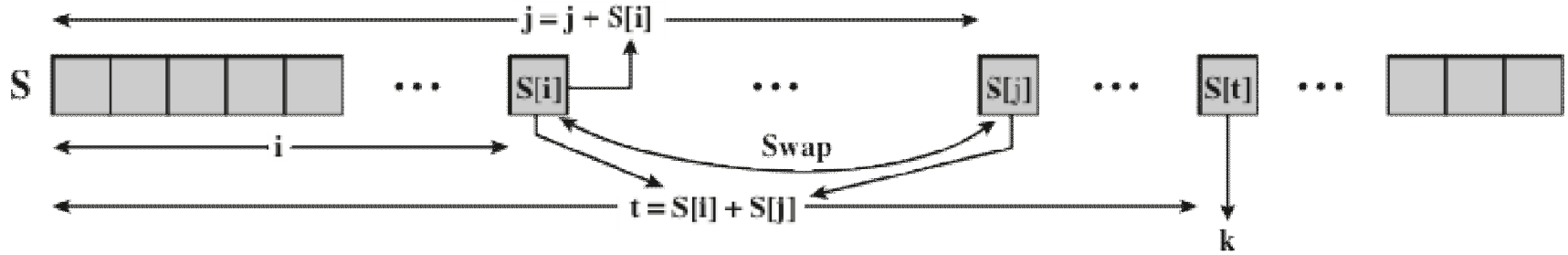
- Designed in 1987 by Ron Rivest for RSA Security
- Variable key size stream cipher with byte-oriented operations
- Based on the use of a random permutation
- Eight to sixteen machine operations are required per output byte and the cipher can be expected to run very quickly in software
- Used in the Secure Sockets Layer/Transport Layer Security (SSL/TLS) standards that have been defined for communication between Web browsers and servers
- Is also used in the Wired Equivalent Privacy (WEP) protocol and the newer WiFi Protected Access (WPA) protocol that are part of the IEEE 802.11 wireless LAN standard



(a) Initial state of S and T



(b) Initial permutation of S

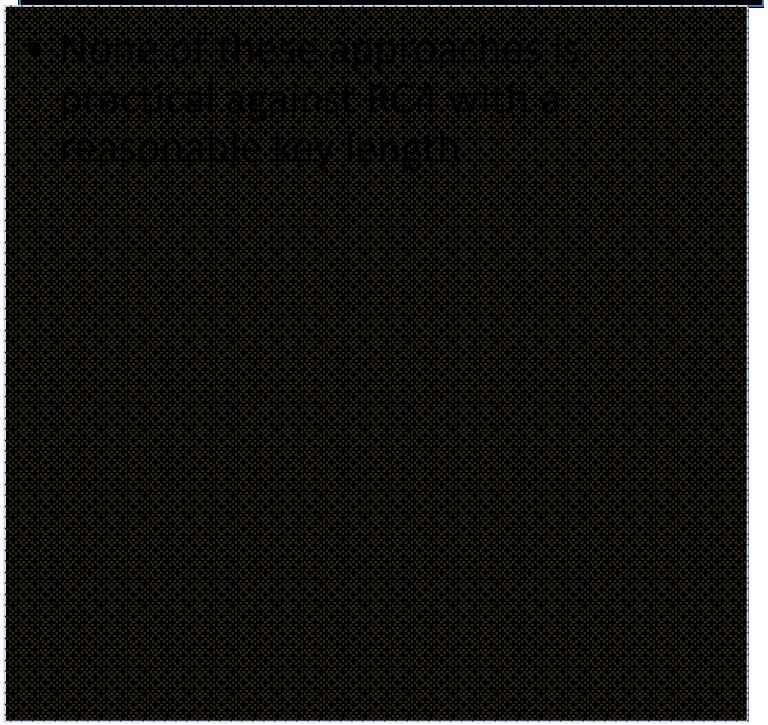


(c) Stream Generation

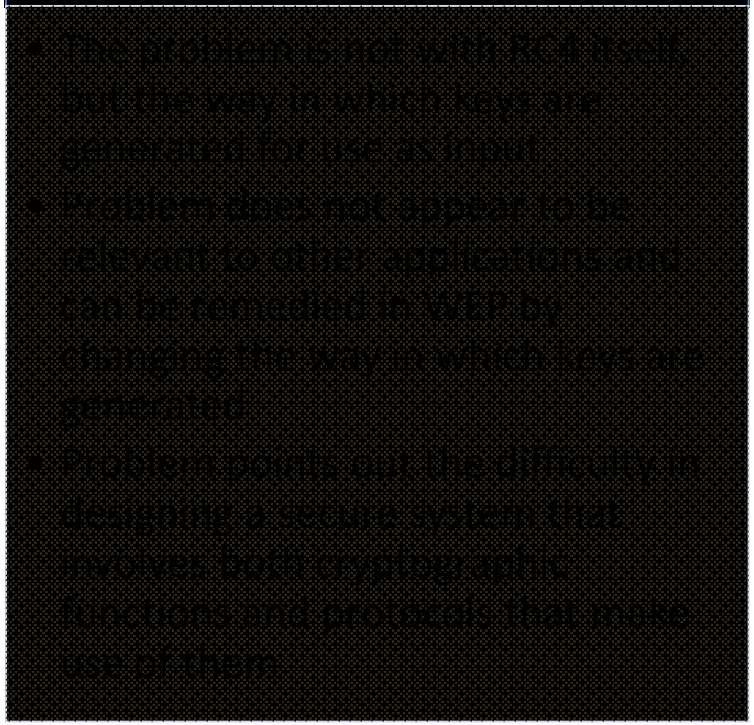
Figure 7.8 RC4

Strength of RC4

A number of papers have been published analyzing methods of attacking RC4



A more serious problem is that the WEP protocol intended to provide confidentiality on 802.11 wireless LAN networks is vulnerable to a particular attack approach

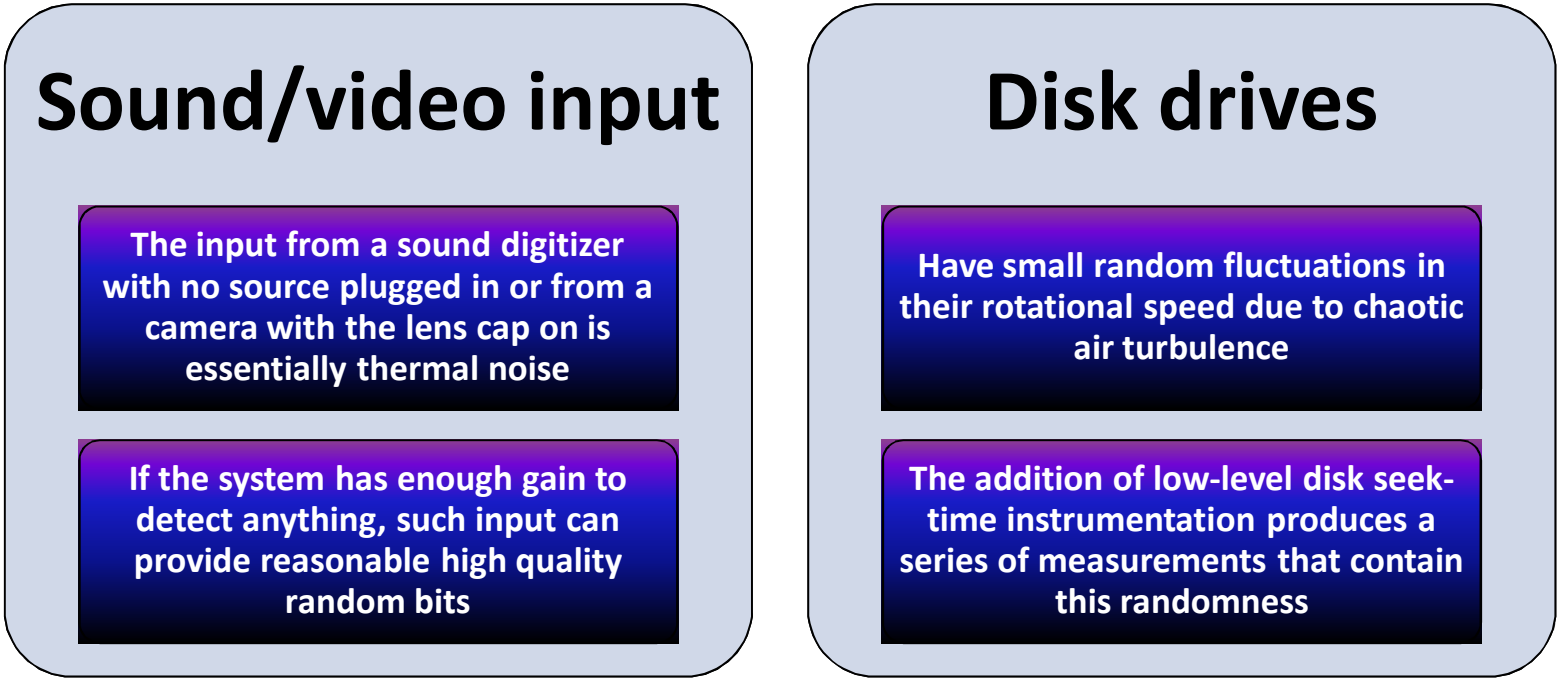


Entropy Sources

- A true random number generator (TRNG) uses a nondeterministic source to produce randomness
- Most operate by measuring unpredictable natural processes such as pulse detectors of ionizing radiation events, gas discharge tubes, and leaky capacitors
- Intel has developed a commercially available chip that samples thermal noise by amplifying the voltage measured across undriven resistors
- LavaRnd is an open source project for creating truly random numbers using inexpensive cameras, open source code, and inexpensive hardware
  - The system uses a saturated CCD in a light-tight can as a chaotic source to produce the seed; software processes the result into truly random numbers in a variety of formats

Possible Sources of Randomness

RFC 4086 lists the following possible sources of randomness that can be used on a computer to generate true random sequences:



There is also an online service (http://www.random.org/) which can deliver random sequences securely over the Internet.

Table 7.5

	Pseudorandom Number Generators	True Random Number Generators
Efficiency	Very efficient	Generally inefficient
Determinism	Deterministic	Nondeterministic
Periodicity	Periodic	Aperiodic

Comparison of PRNGs and TRNGs

Skew

- A TRNG may produce an output that is biased in some way, such as having more ones than zeros or vice versa
  - Deskewing algorithms
    - Methods of modifying a bit stream to reduce or eliminate the bias
    - One approach is to pass the bit stream through a hash function such as MD5 or SHA-1
    - RFC 4086 recommends collecting input from multiple hardware sources and then mixing these using a hash function to produce random output
- Operating systems typically provide a built-in mechanism for generating random numbers
  - Linux uses four entropy sources: mouse and keyboard activity, disk I/O operations, and specific interrupts
    - Bits are generated from these four sources and combined in a pooled buffer
    - When random bits are needed the appropriate number of bits are read from the buffer and passed through the SHA-1 hash function

Intel Digital Random Number Generator

- TRNGs have traditionally been used only for key generation and other applications where only a small number of random bits were required
  - This is because TRNGs have generally been inefficient with a low bit rate of random bit production
- The first commercially available TRNG that achieves bit production rates comparable with that of PRNGs is the Intel digital random number generator offered on new multicore chips since May 2012
  - It is implemented entirely in hardware
  - The entire DRNG is on the same multicore chip as the processors

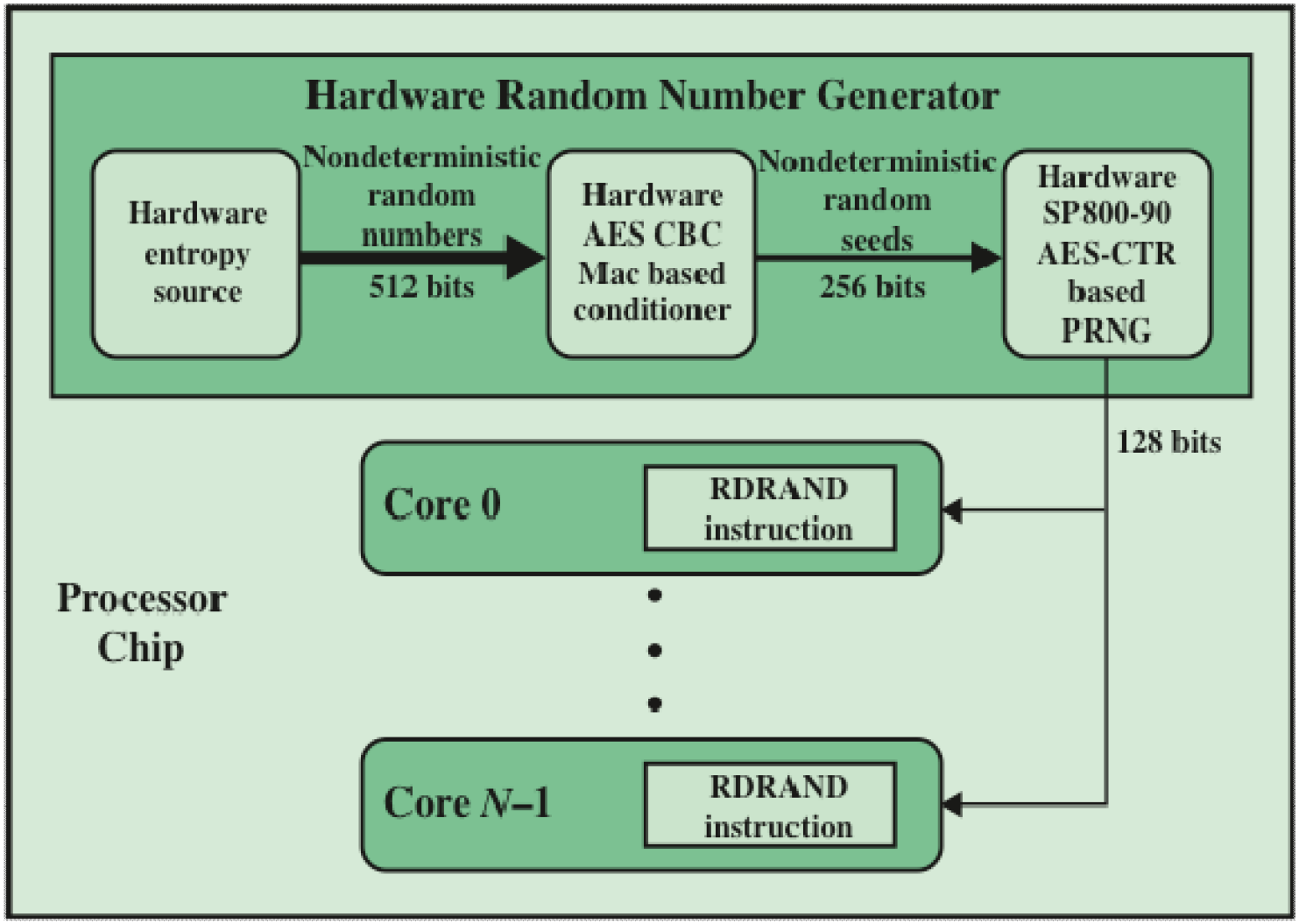


Figure 7.9 Intel Processor Chip with Random Number Generator

Intel DRNG Logical Structure

(Figure 7.10 is located on page 226 in textbook)

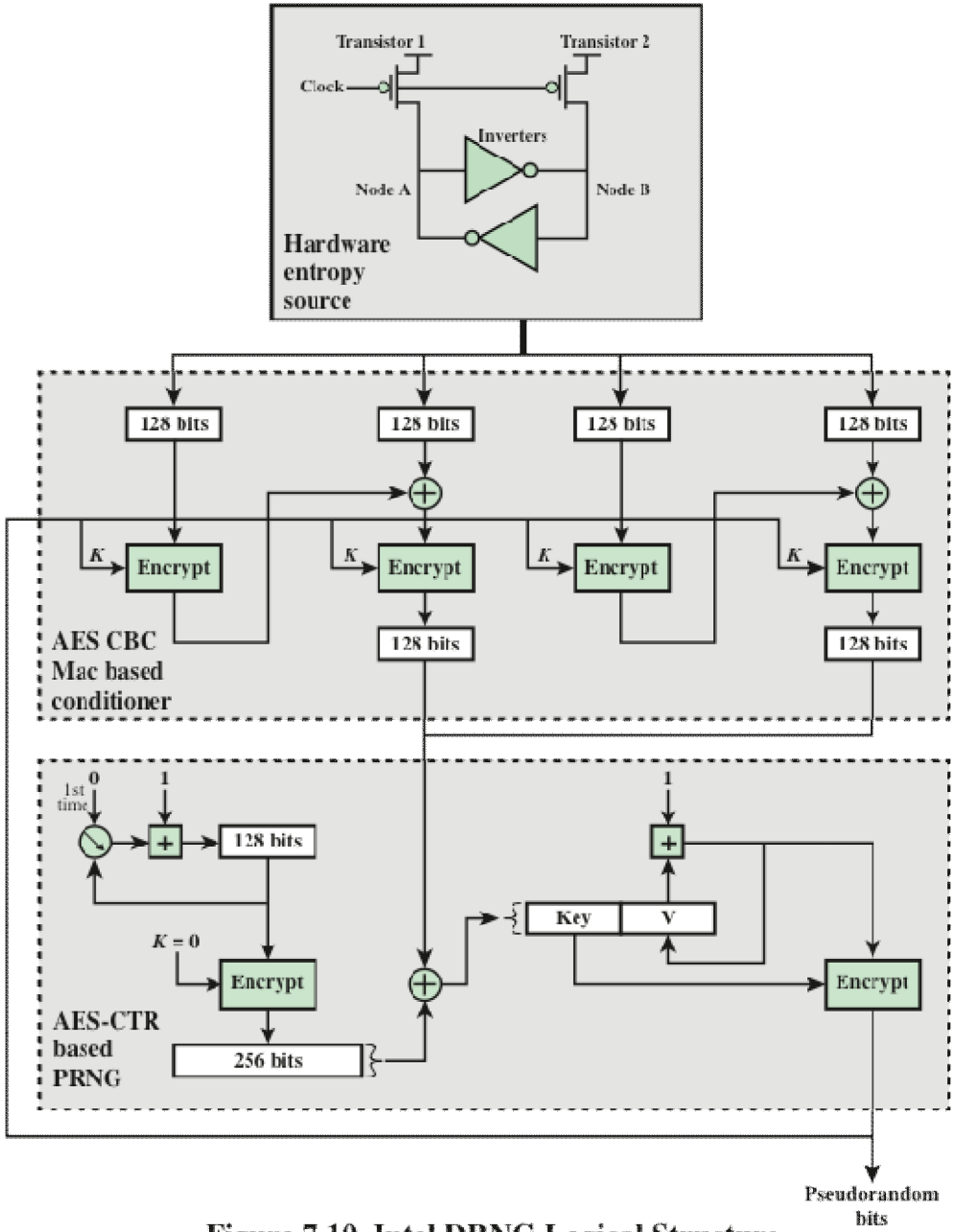
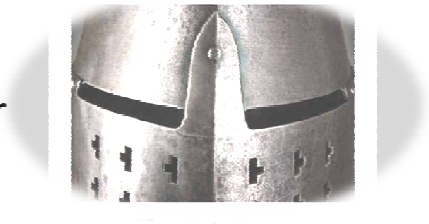


Figure 7.10 Intel DRNG Logical Structure

Summary

- Principles of pseudorandom number generation
  - The use of random numbers
  - TRNGs, PRNGs, and PRFs
  - PRNG requirements
  - Algorithm design
- Pseudorandom number generators
  - Linear congruential generators
  - Blum Blum Shub generator
- Pseudorandom number generation using a block cipher
  - PRNG using block cipher modes of operation
  - ANSI X9.17 PRNG
  - NIST CTR_DRBG



- Stream ciphers
- RC4
  - Initialization of S
  - Stream generation
  - Strength of RC4
- True random number generators
  - Entropy sources
  - Comparison of PRNGs and TRNGs
  - Skew
  - Intel digital random number generator
  - DRNG hardware architecture
  - DRNG logical structure